Universidad Mariano Gálvez de Guatemala

Sede de San Pedro Sacatepéquez, San Marcos.



Informe

Documentación integral del proyecto Tienda Bonita

Universidad Mariano Gálvez de Guatemala

Sede de San Pedro Sacatepéquez, San Marcos.

Curso: Programación II

Fecha de entrega: sábado 31 de octubre de 2026

Integrantes:

Nombre	Carne
Rebeca Alejandra Estrada Navarro	0903-25-23194
Carol Andrea Gómez Dionicio	0903-25-23190
Franshesca Sherlyn Cordon Orozco	0903-25-19546
Zashary Adaly Fuentes Fuentes	0903-25-5941
Angel Rodolfo Roblero Palma	0903-25-9479

Índice

1. Introducción 5
2. Objetivos 6
2.1. Objetivo general 6
2.2. Objetivos específicos 6
3. Enlaces a los entregables 6
4. Descripción del diagrama de clases UML 7
4.1. Estructura de entidades y responsabilidades 7
4.1.1. Módulo de control de acceso 7
4.1.2. Módulo de procesos de venta 7
4.1.3. Módulo de gestión de inventario y productos 8
4.2. Criterios y convenciones UML aplicadas 8
5. Descripción del prototipo de pantallas (Wireframe) 10
5.1. Pantalla 1: Inicio de sesión (Login) 10
5.2. Pantalla 2: Punto de venta (Registro de ventas) 10
5.3. Pantalla 3: Gestión de inventario 11
6. Relación entre el diagrama de clases y el prototipo 11
Diccionario de Datos 12
1. Inventario de tablas 12
2. Diccionario por tabla 12
2.1. Tabla: categoría 12
2.2. Tabla: cliente 12
2.3. Tabla: detalle_venta 13
2.4. Tabla: producto 13
2.5. Tabla: producto_importacion 14
2.6. Tabla: proveedor 14
2.7. Tabla: rol 14
2.8. Tabla: usuario 15
2.9. Tabla: venta 15
3. Relaciones entre tablas. 16
Diagrama entidad-relación conceptual 16
Diagrama relacional fuente draw.io 18
Capturas complementarias del manual de usuario 19
Capturas complementarias del manual técnico 28
Capturas complementarias del diccionario de datos 42

Enlaces y archivos de entrega 46
Videos individuales de demostración CRUD 47
Diccionario de datos en texto 48
Tabla categoria 48
Tabla cliente 48
Tabla detalle_venta 49
Tabla producto 49
Tabla producto_importacion 50
Tabla proveedor 50
Tabla rol 51
Tabla usuario 51
Tabla venta 51
Conclusiones 52

1. Introducción

El presente informe técnico tiene como finalidad documentar el proceso de modelado orientado a objetos y el diseño de la interfaz de usuario del sistema informático denominado **TIENDA BONITA**.

El desarrollo de esta propuesta contempla dos elementos fundamentales: la elaboración de un diagrama de clases utilizando el Lenguaje Unificado de Modelado (UML) y la creación de un prototipo de pantallas, mediante los cuales se representa tanto la estructura lógica del sistema como la interacción visual entre el usuario y las principales funcionalidades de la aplicación.

El diagrama de clases permite identificar las entidades principales, sus atributos, métodos, responsabilidades y relaciones, proporcionando una representación estructural del sistema. Por otra parte, el prototipo de interfaz permite visualizar la distribución de los elementos gráficos y los procesos que realizará el usuario durante la utilización de la aplicación.

2. Objetivos

2.1. Objetivo general

Diseñar y documentar la estructura orientada a objetos y la interfaz visual del sistema TIENDA BONITA, mediante la elaboración de un diagrama de clases UML y un prototipo de pantallas que representen las principales funcionalidades de la aplicación.

2.2. Objetivos específicos

- Identificar las entidades principales que intervienen en los procesos de gestión de usuarios, productos, inventario, clientes y ventas.
- Definir los atributos, métodos y responsabilidades de las clases que conforman el modelo del sistema.
- Establecer las relaciones y multiplicidades entre las entidades mediante las convenciones del lenguaje UML.
- Diseñar un prototipo de interfaz que represente las principales pantallas y procesos de interacción del usuario.
- Mantener la correspondencia entre el modelo de clases y los elementos representados en el prototipo visual.

3. Enlaces a los entregables

Los entregables correspondientes al diagrama de clases UML y al prototipo de pantallas se encuentran integrados en un mismo archivo de Draw.io, organizados en pestañas independientes para facilitar su consulta y revisión.

Enlace al archivo de Draw.io (Diagrama de clases y prototipo):

https://drive.google.com/file/d/1IDXmLMw4x9VygN7EQpX3RhLVzzvdaW9Z/view?usp=sharing

El archivo contiene los dos componentes principales del diseño:

- **Diagrama de clases UML:** representación de la estructura estática del sistema, sus entidades, atributos, métodos y relaciones.
- **Prototipo de pantallas:** representación visual de las interfaces principales y de los procesos de interacción del usuario.

4. Descripción del diagrama de clases UML

El diagrama de clases del sistema TIENDA BONITA representa la estructura estática de la aplicación mediante la identificación de las entidades principales que intervienen en los procesos comerciales y administrativos de la tienda.

Las clases se organizan de acuerdo con sus responsabilidades y relaciones, permitiendo representar la lógica del negocio y la manera en que los diferentes componentes se relacionan entre sí.

4.1. Estructura de entidades y responsabilidades

Para facilitar su comprensión, las clases se agrupan en los siguientes módulos:

4.1.1. Módulo de control de acceso

Este módulo administra la información relacionada con los usuarios y sus respectivos roles dentro del sistema.

- **Rol:** representa los perfiles o tipos de acceso disponibles, permitiendo identificar las funciones que corresponden a cada usuario.
- **Usuario:** almacena la información de las cuentas registradas y los datos necesarios para realizar el proceso de autenticación.

Relación: un rol puede estar asociado con cero o varios usuarios, mientras que cada usuario pertenece a un rol, según la multiplicidad representada en el modelo.

4.1.2. Módulo de procesos de venta

Este módulo representa las entidades involucradas en el registro y administración de las operaciones comerciales.

- **Cliente:** representa a las personas registradas en el sistema que realizan compras en la tienda.
- **Venta:** representa una transacción comercial realizada por un cliente. Incluye operaciones como calcularTotal() y generarPDF(), destinadas al cálculo del importe de la venta y la generación de su comprobante.
- **DetalleVenta:** almacena el desglose de los productos incluidos en una venta, considerando información como el producto, la cantidad y los valores correspondientes.

Relaciones:

- Un cliente puede realizar cero o varias ventas.
- Una venta está asociada a un cliente.
- Una venta contiene uno o varios registros de detalle de venta, de acuerdo con la multiplicidad definida en el diagrama.
- Cada detalle de venta pertenece a una venta específica.

La relación entre Venta y DetalleVenta se representa mediante composición, debido a que los detalles forman parte de la estructura de una venta.

4.1.3. Módulo de gestión de inventario y productos

Este módulo contempla las entidades encargadas de administrar los productos y controlar sus existencias.

- **Producto:** representa los artículos disponibles para la comercialización. Contiene los atributos relacionados con su identificación, descripción, precio y existencias. También contempla operaciones como esStockBajo() y estaVencido() para evaluar las condiciones del inventario.
- **Categoria:** permite clasificar los productos de acuerdo con sus características.
- **Proveedor:** representa a las personas o empresas encargadas de suministrar los productos.
- **MovimientoInventario:** registra las operaciones relacionadas con las entradas y salidas de existencias, permitiendo llevar un control de los movimientos realizados.
- **Devolucion:** representa los procesos de devolución de productos y su relación con las operaciones comerciales correspondientes.

Relaciones:

- Una categoría puede estar asociada con varios productos.
- Un proveedor puede suministrar varios productos.
- Los movimientos de inventario permiten registrar cambios en las existencias de los productos.
- Las devoluciones se relacionan con los procesos de venta y los productos correspondientes, según las relaciones definidas en el modelo.

4.2. Criterios y convenciones UML aplicadas

Para representar correctamente la estructura del sistema se aplicaron las siguientes convenciones del Lenguaje Unificado de Modelado (UML):

Encapsulamiento:

Los atributos de las clases se representan con visibilidad privada (-), con el propósito de proteger la información interna de cada entidad. El acceso y modificación de estos atributos se realiza mediante métodos públicos (+), como getters y setters, cuando corresponde.

Visibilidad de miembros:

Se utilizan los símbolos UML para identificar el nivel de acceso de los atributos y métodos:

- + Público.
- - Privado.
- # Protegido.

Multiplicidad:

Se establecen cardinalidades para representar la cantidad de objetos que pueden participar en una relación:

- 1: representa una única instancia obligatoria.
- 0..*: representa cero o múltiples instancias.
- 1..*: representa una o múltiples instancias.
- 0..1: representa una instancia opcional.

Relaciones entre clases:

Se utilizan asociaciones y relaciones de composición para representar la dependencia y conexión entre las entidades del sistema. Estas relaciones permiten comprender cómo se organiza la información y cómo interactúan los diferentes componentes.

Consistencia del modelo:

Se busca mantener correspondencia entre los atributos, métodos, responsabilidades y relaciones de las clases, de manera que el diseño represente adecuadamente los procesos principales del sistema.

5. Descripción del prototipo de pantallas (Wireframe)

El prototipo de interfaz de usuario representa visualmente las principales pantallas del sistema TIENDA BONITA. Su propósito es mostrar cómo se organizarán los elementos gráficos y cómo interactuará el usuario con las funcionalidades principales de la aplicación.

El diseño contempla tres vistas fundamentales que representan los procesos de acceso, registro de ventas y administración del inventario.

5.1. Pantalla 1: Inicio de sesión (Login)

Esta pantalla permite al usuario ingresar al sistema mediante sus credenciales de acceso.

Su diseño contempla los campos necesarios para introducir el nombre de usuario y la contraseña, así como el botón correspondiente para iniciar sesión.

La interfaz representa el proceso de autenticación relacionado con la entidad Usuario y permite considerar el rol asociado a cada cuenta para controlar el acceso a las funcionalidades del sistema.

Funcionalidad principal: permitir el acceso seguro de los usuarios registrados.

5.2. Pantalla 2: Punto de venta (Registro de ventas)

Esta pantalla representa el proceso de registro de las operaciones comerciales realizadas en la tienda.

El prototipo contempla los siguientes elementos:

- Selección del cliente que realiza la compra.
- Búsqueda y selección de productos.
- Ingreso de las cantidades correspondientes.
- Incorporación de varios productos al detalle de la venta.
- Visualización de los productos seleccionados en una tabla.
- Presentación de subtotales y cálculo del total de la operación.
- Opción para generar el comprobante de venta en formato PDF.

Esta interfaz se relaciona directamente con las entidades Cliente, Venta, Producto y DetalleVenta.

Funcionalidad principal: representar el proceso de selección de productos y registro de una venta.

5.3. Pantalla 3: Gestión de inventario

Esta pantalla representa el panel destinado a la administración y consulta de los productos registrados en el sistema.

El diseño contempla las siguientes funcionalidades:

- Búsqueda de productos registrados.
- Visualización de información y existencias.
- Registro de nuevos productos.
- Consulta y actualización de información.
- Identificación de productos con existencias bajas.
- Visualización de alertas relacionadas con productos próximos a vencer.

Esta interfaz se relaciona principalmente con las entidades Producto, Categoria, Proveedor y MovimientoInventario.

Funcionalidad principal: facilitar el control y la administración de los productos y sus existencias.

6. Relación entre el diagrama de clases y el prototipo

El diagrama de clases y el prototipo de pantallas representan dos perspectivas complementarias del sistema TIENDA BONITA.

El diagrama UML define la estructura interna de la aplicación, identificando las entidades, atributos, métodos y relaciones necesarias para representar la lógica del negocio.

Por su parte, el prototipo de pantallas representa la interacción visual del usuario con las funcionalidades principales.

Ambos elementos mantienen una correspondencia entre las clases del modelo y los componentes de la interfaz.

Por ejemplo: la pantalla de inicio de sesión se relaciona con la clase Usuario, la pantalla de ventas utiliza las entidades Cliente, Producto, Venta y DetalleVenta, mientras que la pantalla de inventario se relaciona con la clase Producto y las entidades encargadas de su clasificación y control.

Esta relación permite mantener una visión organizada del sistema y facilita el proceso de implementación posterior.

Diccionario de Datos

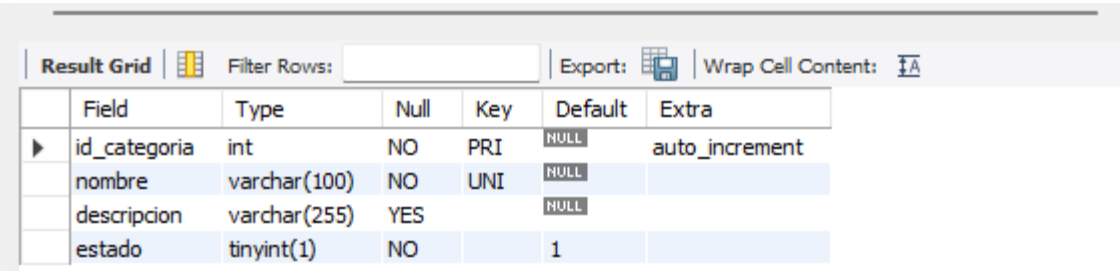
1. Inventario de tablas

Tabla	Finalidad
categoria	Clasificación de productos
cliente	Datos de clientes
detalle_venta	Líneas y productos vendidos
producto	Catálogo y existencias
producto_importacion	Datos auxiliares para importación
proveedor	Datos de proveedores
rol	Perfiles de acceso
usuario	Cuentas de acceso al sistema
venta	Encabezado de transacciones de venta

2. Diccionario por tabla

2.1. Tabla: categoría

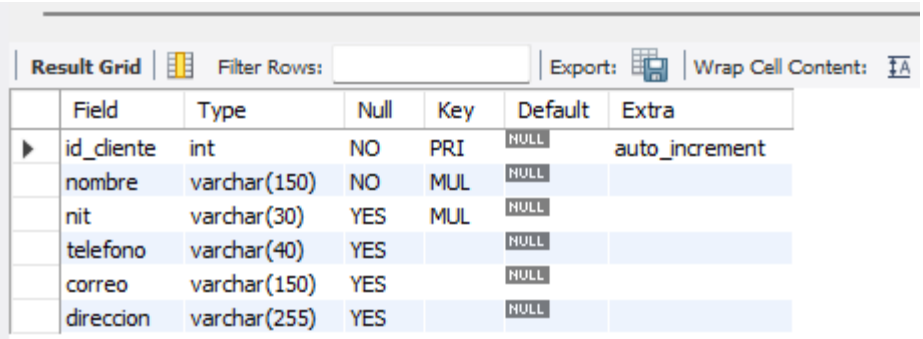
Clasificación de productos



Field	Type	Null	Key	Default	Extra
id_categoria	int	NO	PRI	NULL	auto_increment
nombre	varchar(100)	NO	UNI	NULL	
descripcion	varchar(255)	YES		NULL	
estado	tinyint(1)	NO		1	

2.2. Tabla: cliente

Datos de clientes

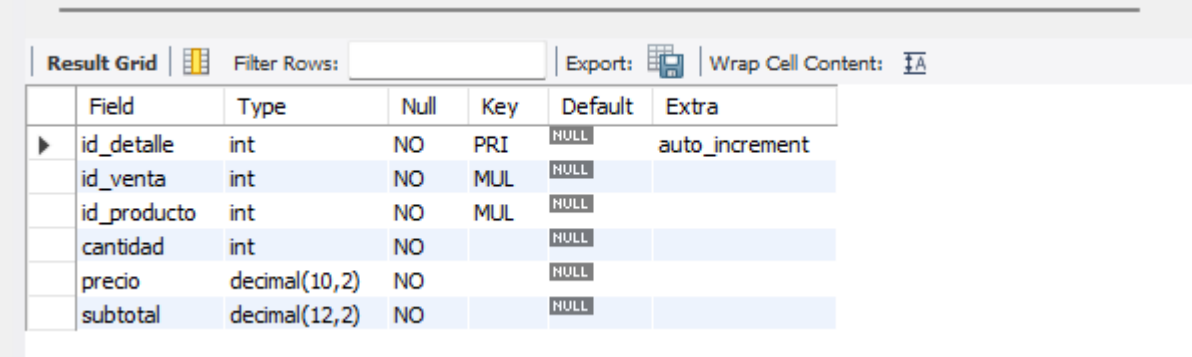


Field	Type	Null	Key	Default	Extra
id_cliente	int	NO	PRI	NULL	auto_increment
nombre	varchar(150)	NO	MUL	NULL	
nit	varchar(30)	YES	MUL	NULL	
telefono	varchar(40)	YES		NULL	
correo	varchar(150)	YES		NULL	
direccion	varchar(255)	YES		NULL	

2.3. Tabla: detalle_venta

Líneas y productos vendidosRelaciones: id_producto → producto.id_producto; id_venta → venta.id_venta.

Restricciones CHECK: chk_detalle_cantidad.



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

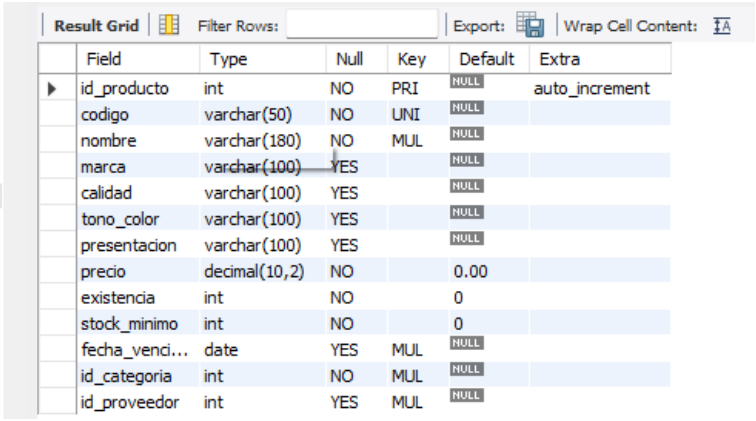
Field	Type	Null	Key	Default	Extra
id_detalle	int	NO	PRI	NULL	auto_increment
id_venta	int	NO	MUL	NULL	
id_producto	int	NO	MUL	NULL	
cantidad	int	NO		NULL	
precio	decimal(10,2)	NO		NULL	
subtotal	decimal(12,2)	NO		NULL	

2.4. Tabla: producto

Catálogo y existencias

Relaciones: id_categoria → categoria.id_categoria; id_proveedor → proveedor.id_proveedor.

Restricciones CHECK: chk_producto_existencia, chk_producto_precio, chk_producto_stock_minimo.



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id_producto	int	NO	PRI	NULL	auto_increment
codigo	varchar(50)	NO	UNI	NULL	
nombre	varchar(180)	NO	MUL	NULL	
marca	varchar(100)	YES		NULL	
calidad	varchar(100)	YES		NULL	
tono_color	varchar(100)	YES		NULL	
presentacion	varchar(100)	YES		NULL	
precio	decimal(10,2)	NO		0.00	
existencia	int	NO		0	
stock_minimo	int	NO		0	
fecha_venci...	date	YES	MUL	NULL	
id_categoria	int	NO	MUL	NULL	
id_proveedor	int	YES	MUL	NULL	

2.5. Tabla: producto_importacion

Datos auxiliares para importación

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
codigo	varchar(50)	YES		NULL	
nombre	varchar(180)	YES		NULL	
marca	varchar(100)	YES		NULL	
calidad	varchar(100)	YES		NULL	
tono_color	varchar(100)	YES		NULL	
presentacion	varchar(100)	YES		NULL	
precio	decimal(10,2)	YES		NULL	
existencia	int	YES		NULL	
stock_minimo	int	YES		NULL	
fecha_vencimiento	varchar(50)	YES		NULL	
id_categoria	int	YES		NULL	
id_proveedor	int	YES		NULL	
extra1	varchar(255)	YES		NULL	
extra2	varchar(255)	YES		NULL	

2.6. Tabla: proveedor

Datos de proveedores

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id_proveedor	int	NO	PRI	NULL	auto_increment
nombre	varchar(150)	NO		NULL	
nit	varchar(30)	YES		NULL	
telefono	varchar(40)	YES		NULL	
correo	varchar(150)	YES		NULL	
direccion	varchar(255)	YES		NULL	

\

2.7. Tabla: rol

Perfiles de acceso

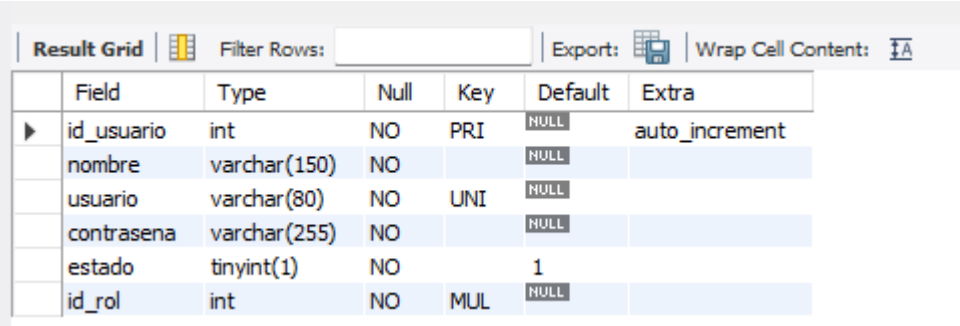
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id_rol	int	NO	PRI	NULL	auto_increment
nombre	varchar(80)	NO	UNI	NULL	
permisos	varchar(255)	YES		NULL	

2.8. Tabla: usuario

Cuentas de acceso al sistema

Relaciones: id_rol → rol.id_rol.

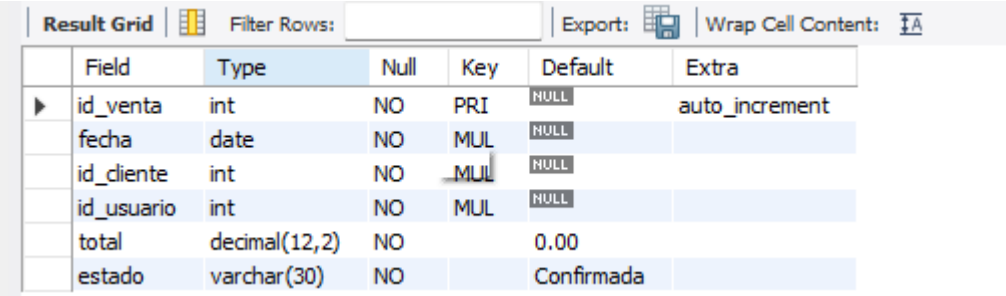


Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id_usuario	int	NO	PRI	NULL	auto_increment
nombre	varchar(150)	NO		NULL	
usuario	varchar(80)	NO	UNI	NULL	
contrasena	varchar(255)	NO		NULL	
estado	tinyint(1)	NO		1	
id_rol	int	NO	MUL	NULL	

2.9. Tabla: venta

Encabezado de transacciones de venta



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id_venta	int	NO	PRI	NULL	auto_increment
fecha	date	NO	MUL	NULL	
id_cliente	int	NO	MUL	NULL	
id_usuario	int	NO	MUL	NULL	
total	decimal(12,2)	NO		0.00	
estado	varchar(30)	NO		Confirmada	

Relaciones: id_cliente → cliente.id_cliente; id_usuario → usuario.id_usuario.

3. Relaciones entre tablas.

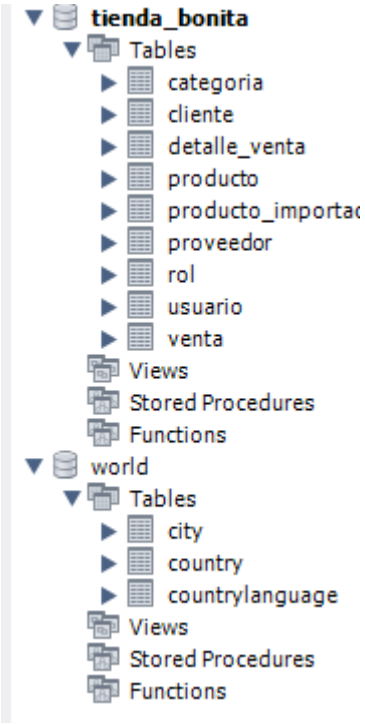


Diagrama entidad-relación conceptual

Material gráfico recibido para documentar las entidades y relaciones del sistema.

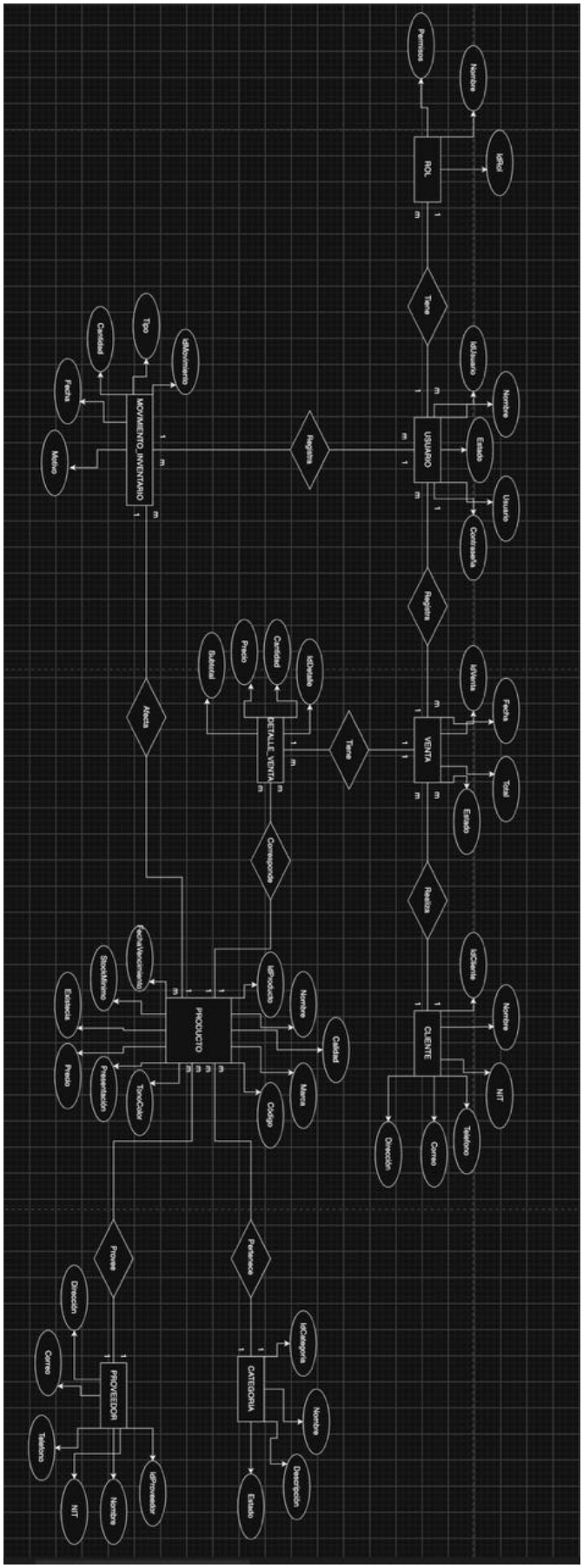
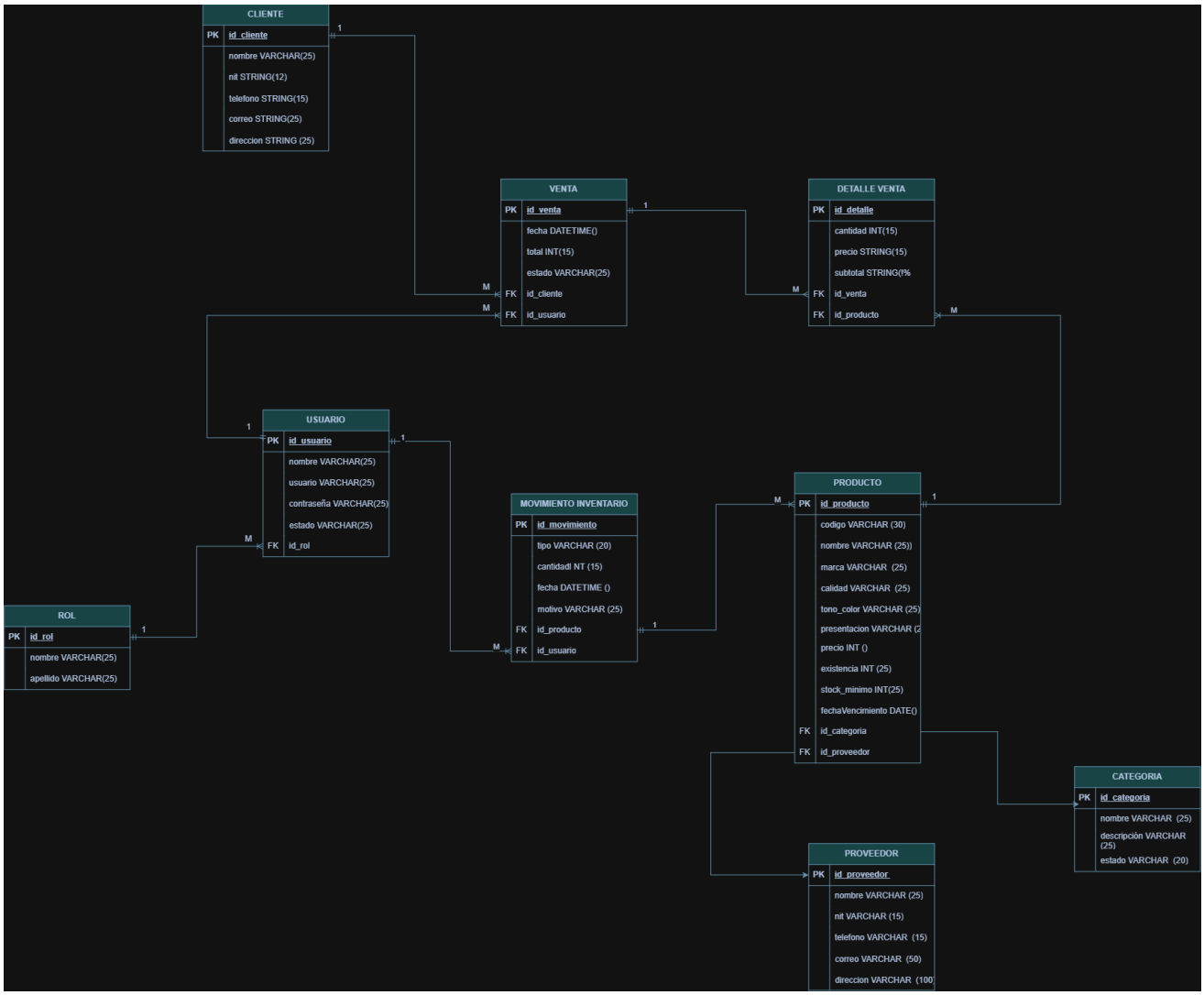
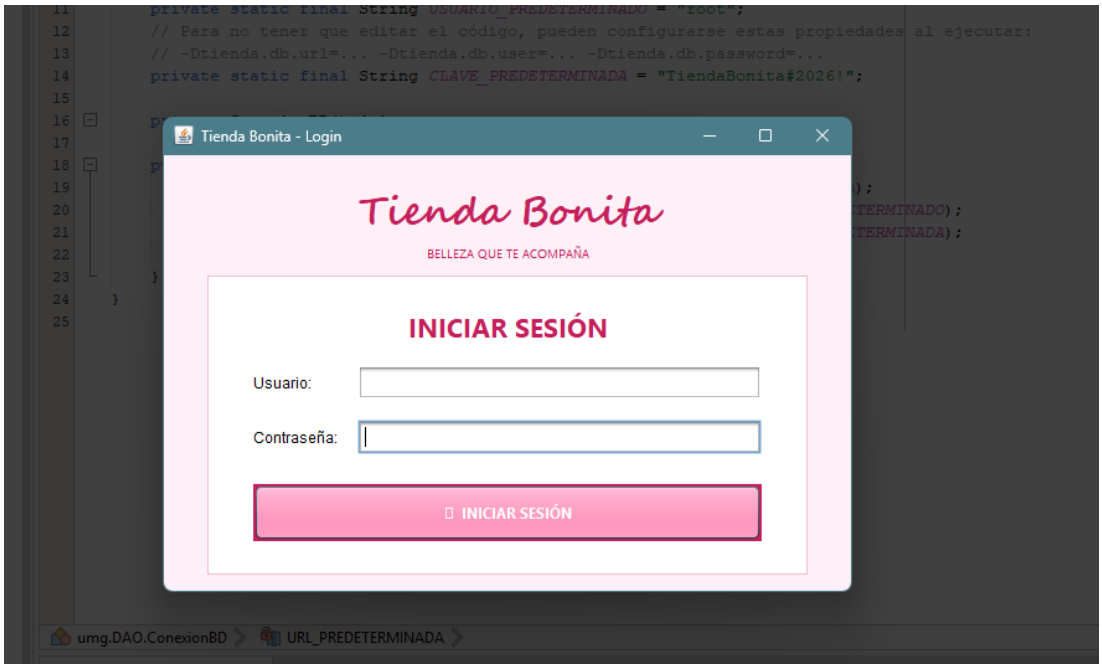

Permisos
Nombre
IdRol
ROL
Tiene
IdUsuario
Nombre
Estado
Usuario
Contraseña
USUARIO
Registra
Registra
IdVenta
Fecha
Total
Estado
VENTA
Tiene
IdDetalle
Cantidad
Precio
Subtotal
DETALLE_VENTA
Corresponde
Realiza
IdCliente
Nombre
NIT
Teléfono
Correo
Dirección
CLIENTE
Afecta
IdMovimiento
Tipo
Cantidad
Fecha
Motivo
MOVIMIENTO_INVENTARIO
IdProducto
Nombre
Cantidad
Marca
Código
TonoColor
Presentación
Precio
Existencia
StockMínimo
FechaVencimiento
PRODUCTO
Provee
Pertenece
IdProveedor
Nombre
NIT
Teléfono
Correo
Dirección
PROVEEDOR
IdCategoría
Nombre
Descripción
Estado
CATEGORIA

Diagrama relacional fuente draw.io



Capturas complementarias del manual de usuario

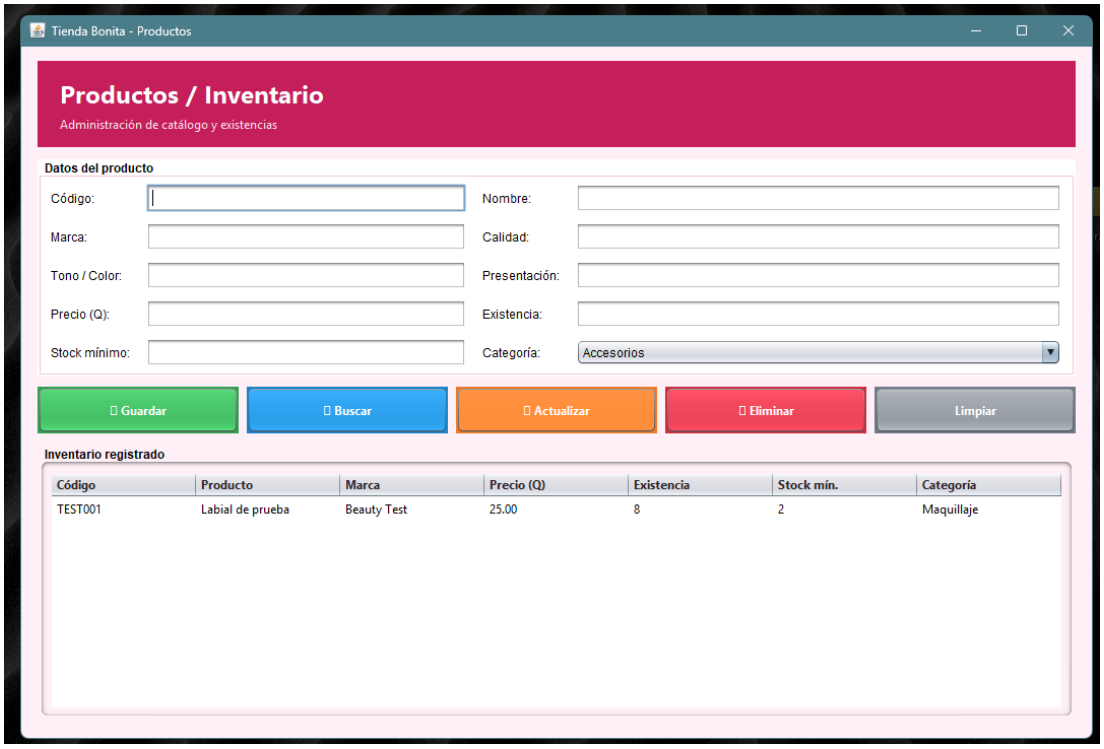
Se conservan las imágenes incrustadas en el documento original para mantener el apoyo visual y los detalles de interfaz o estructura que no aparecen en el texto extraído.



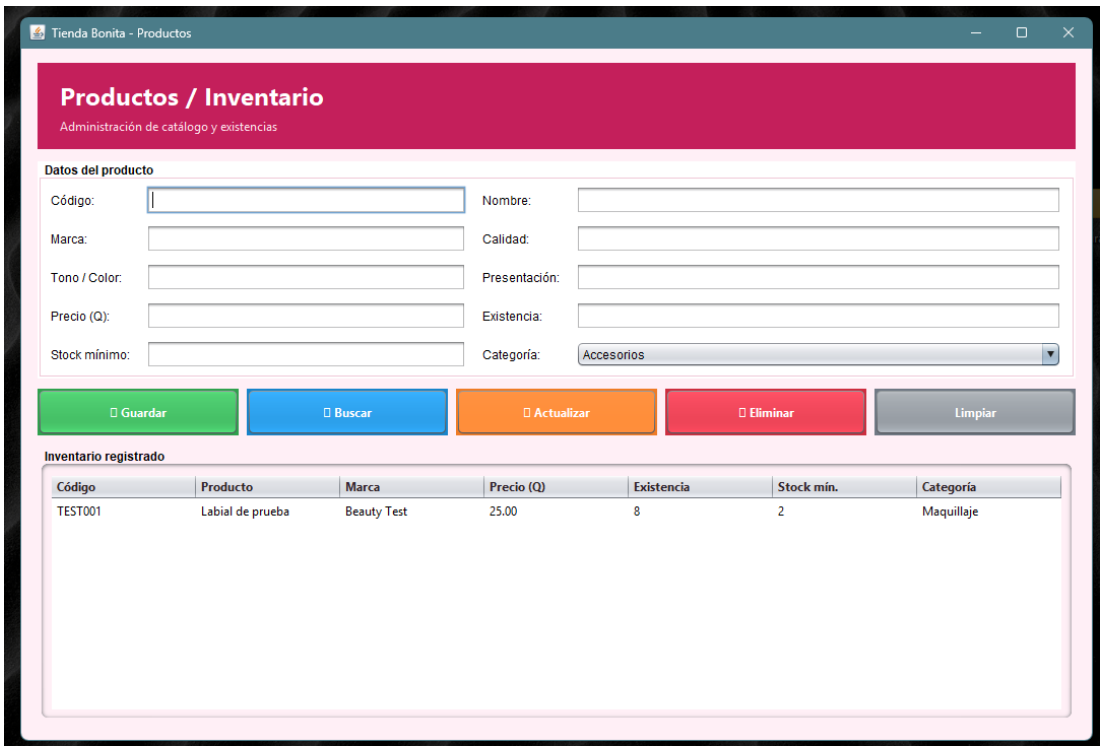
Manual de usuario — captura 1



Manual de usuario — captura 2



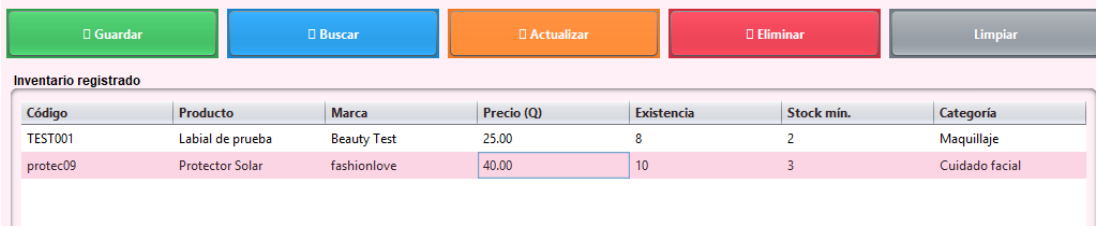
Manual de usuario — captura 3



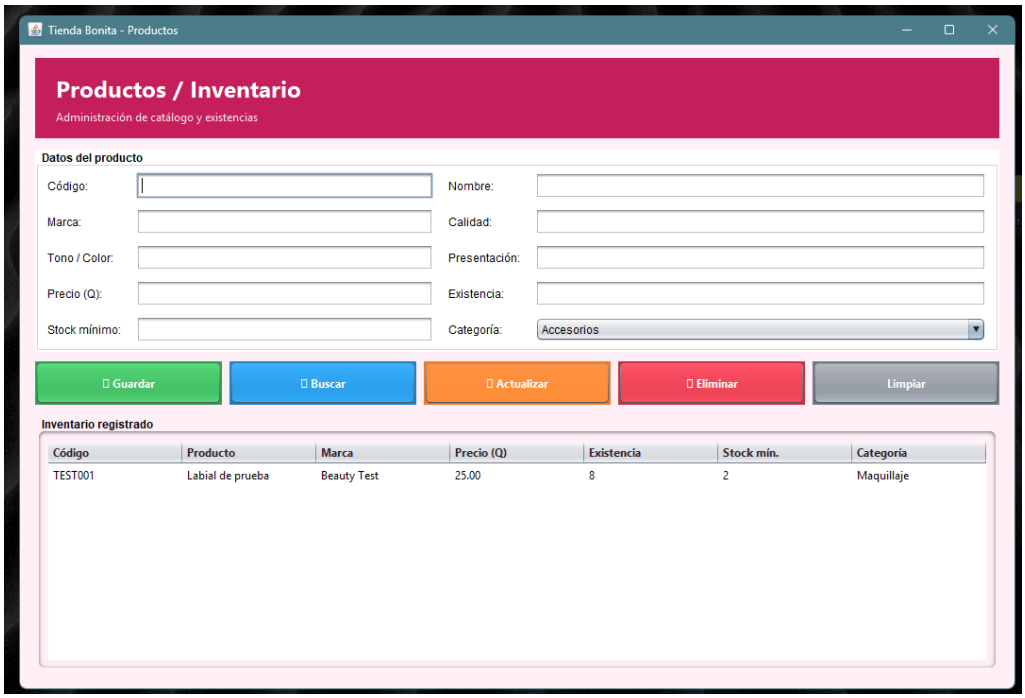
Manual de usuario — captura 4



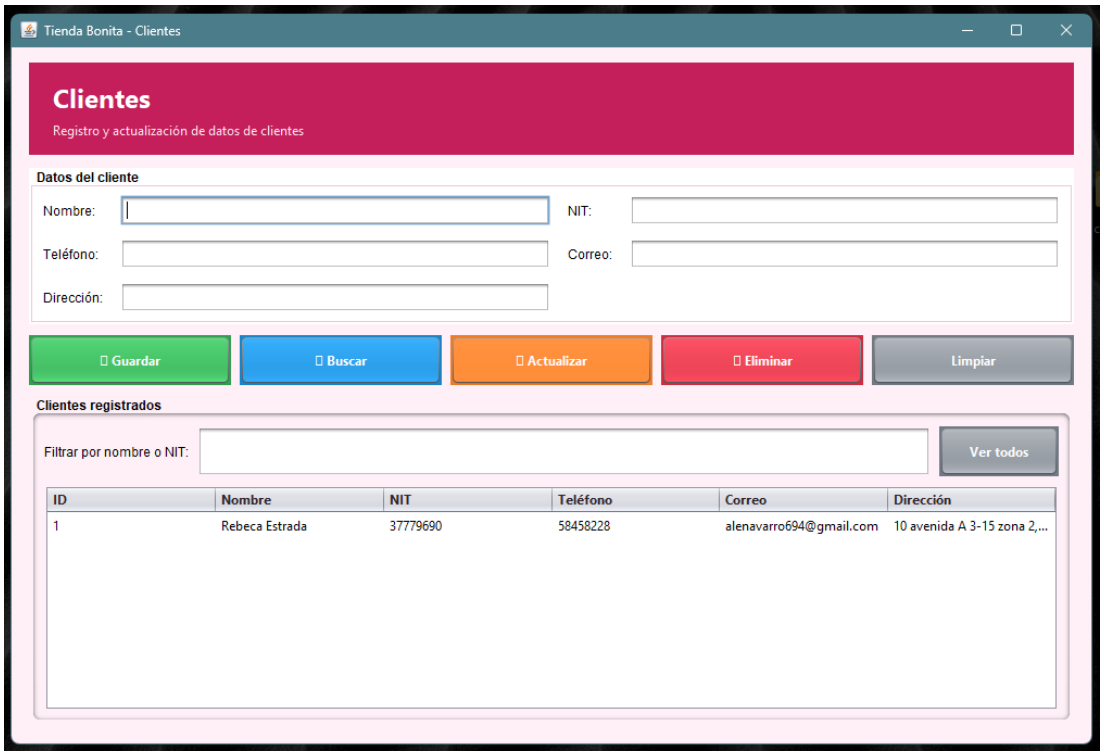
Manual de usuario — captura 5



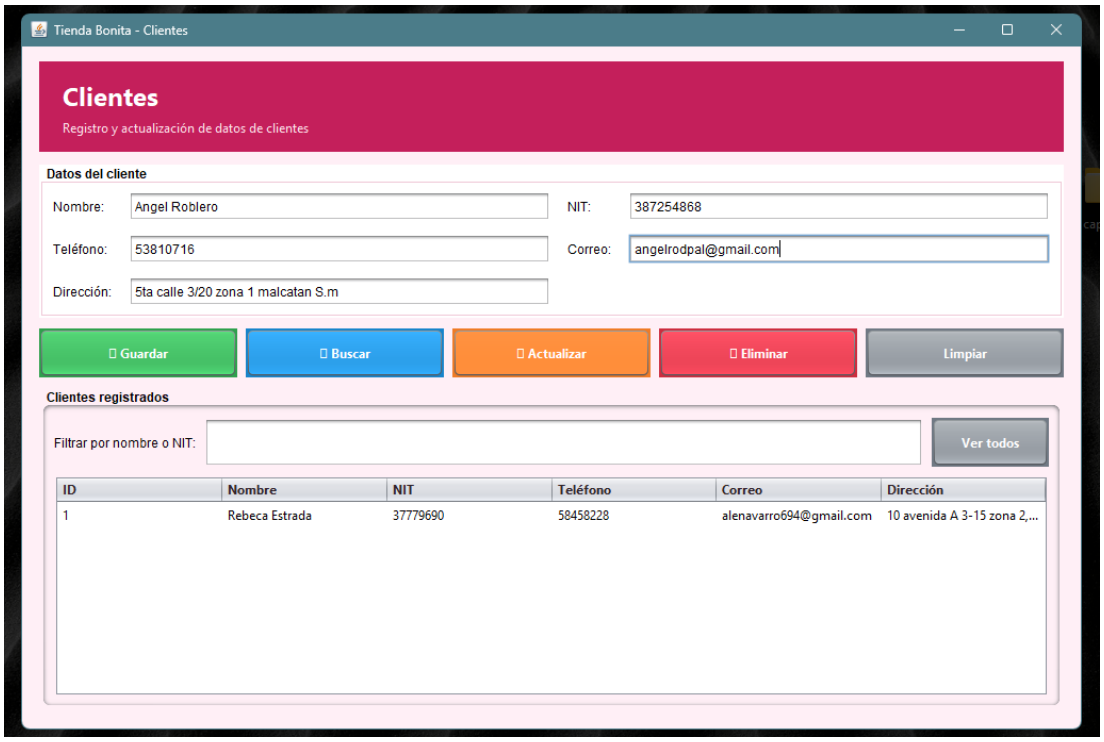
Manual de usuario — captura 6



Manual de usuario — captura 7



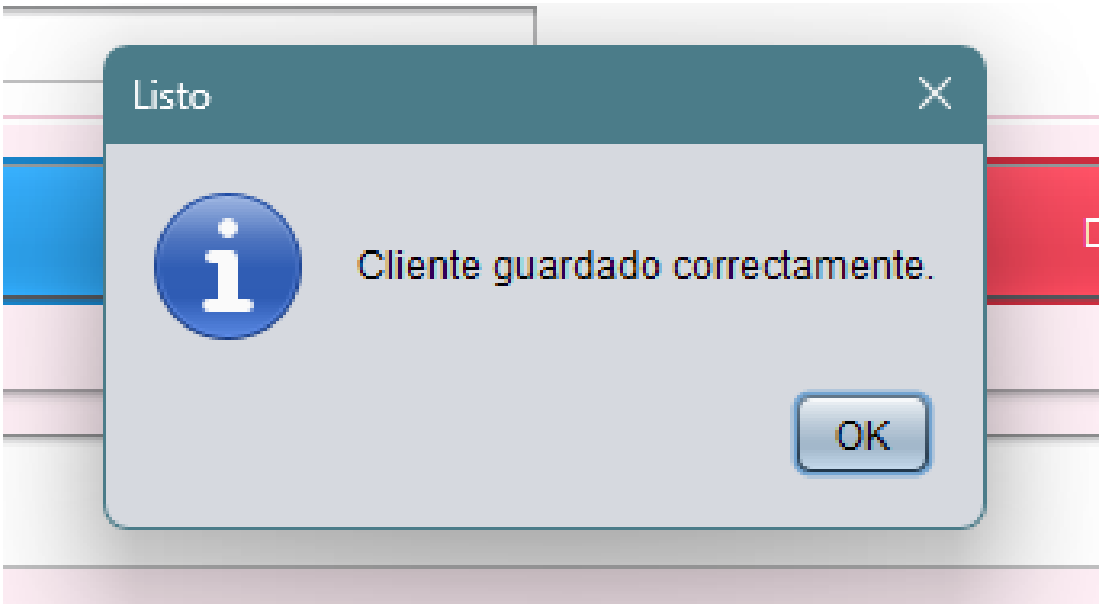
Manual de usuario — captura 8



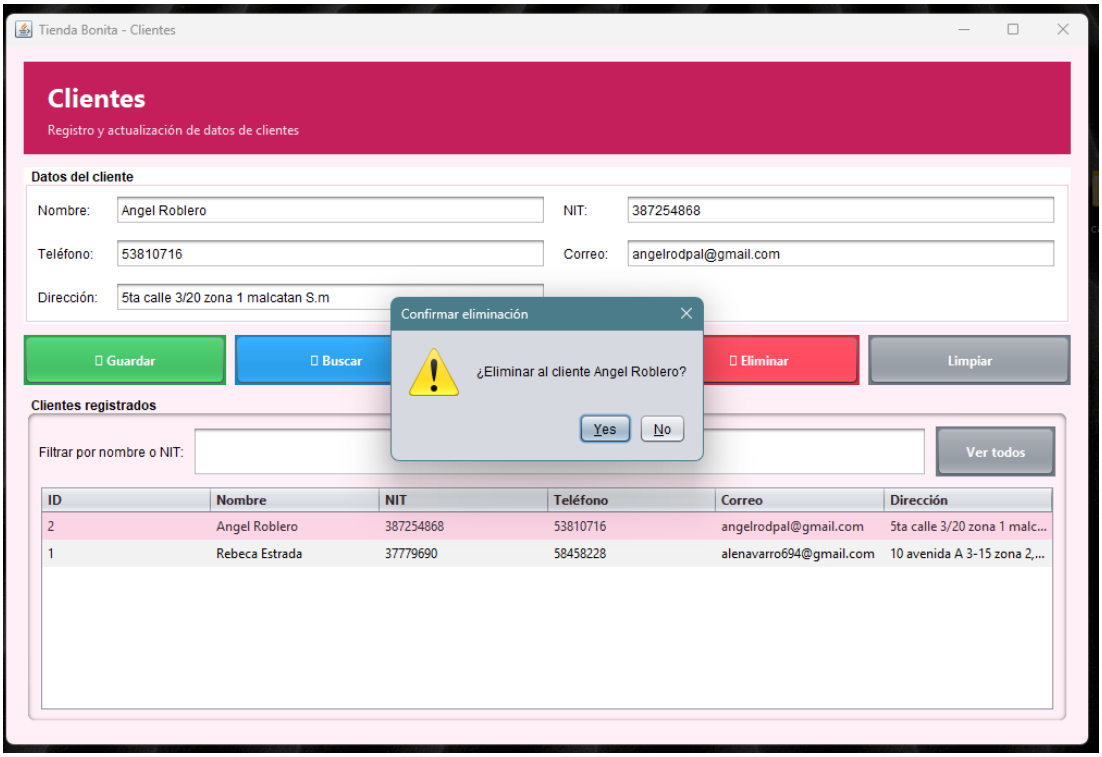
Manual de usuario — captura 9



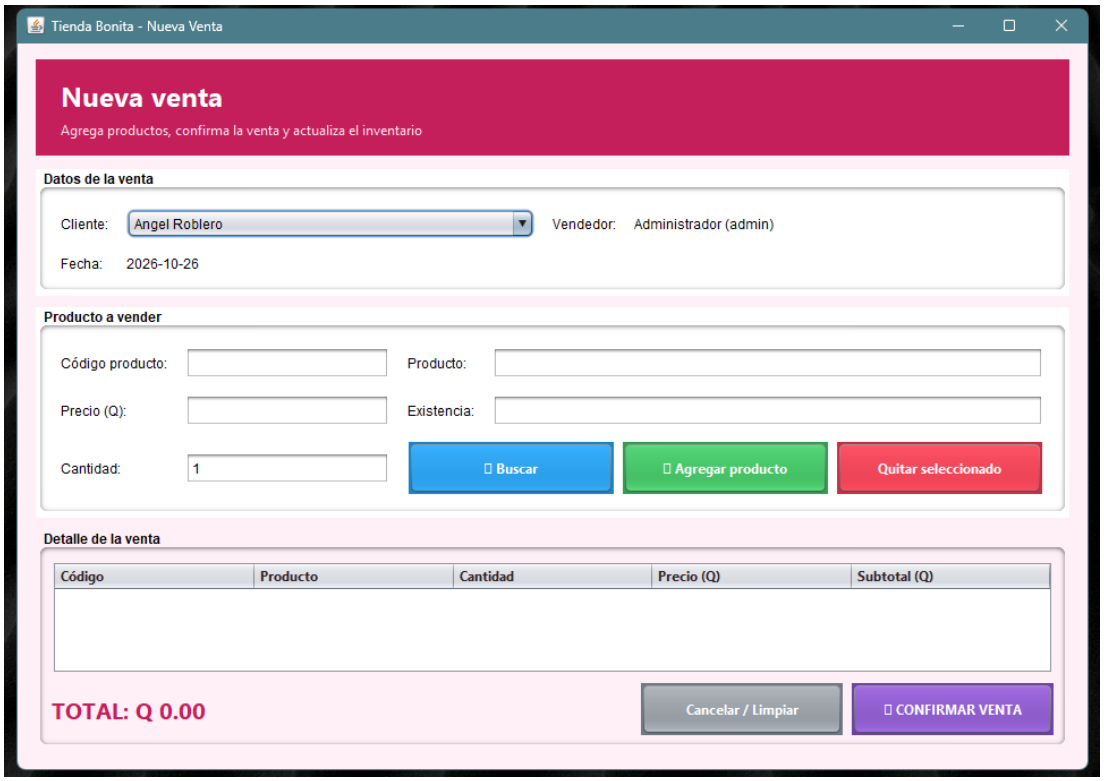
Manual de usuario — captura 10



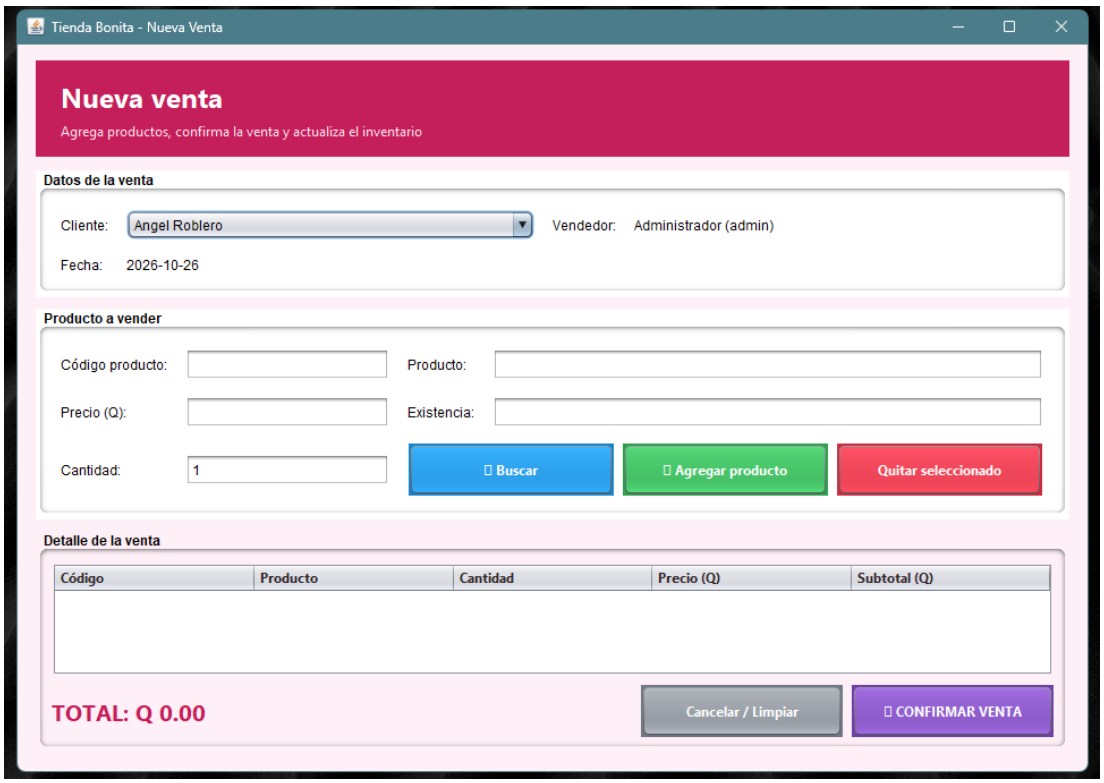
Manual de usuario — captura 11



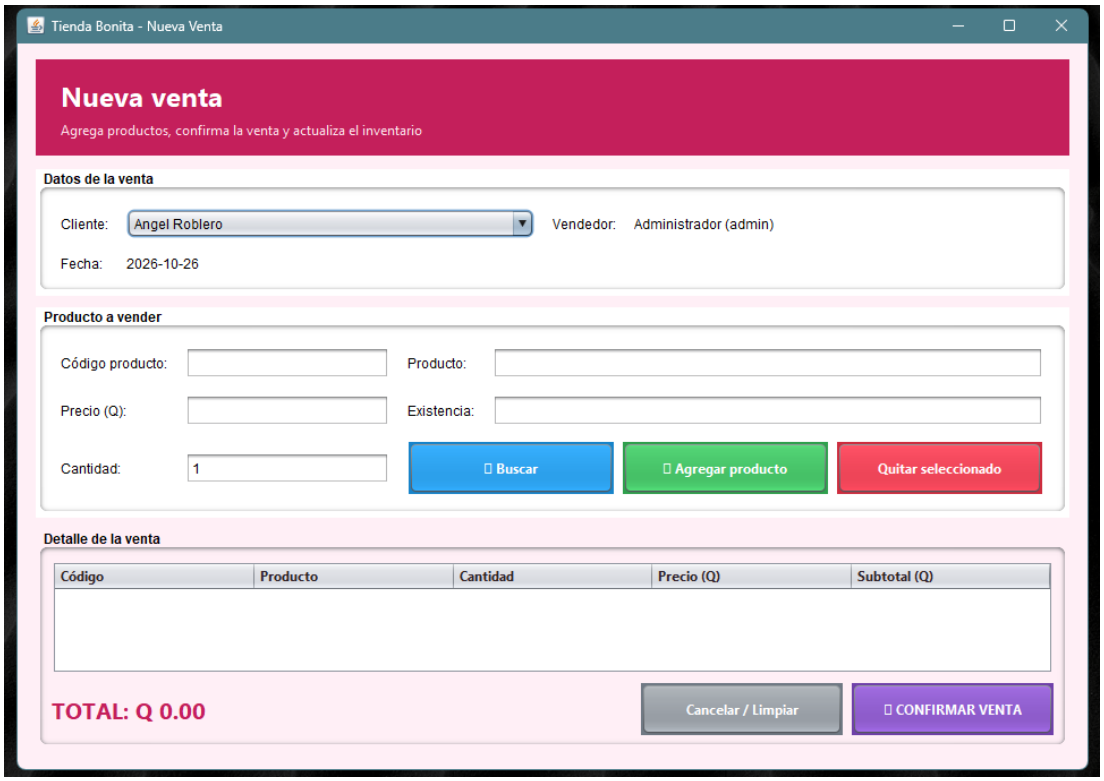
Manual de usuario — captura 12



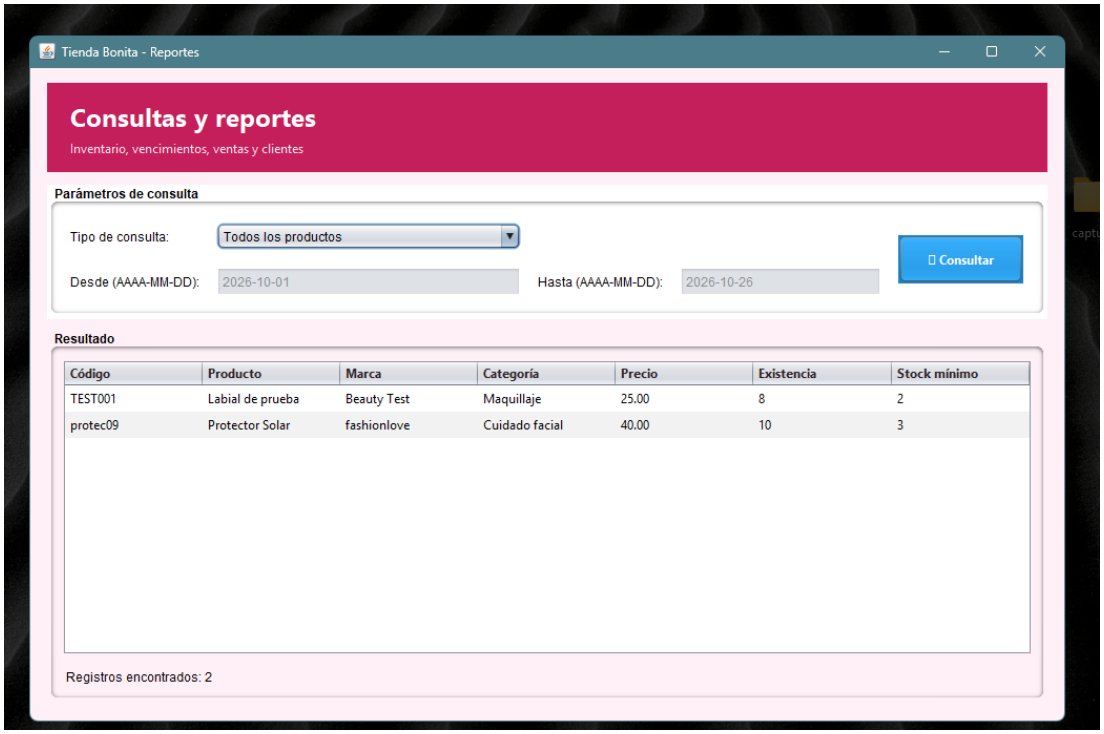
Manual de usuario — captura 13



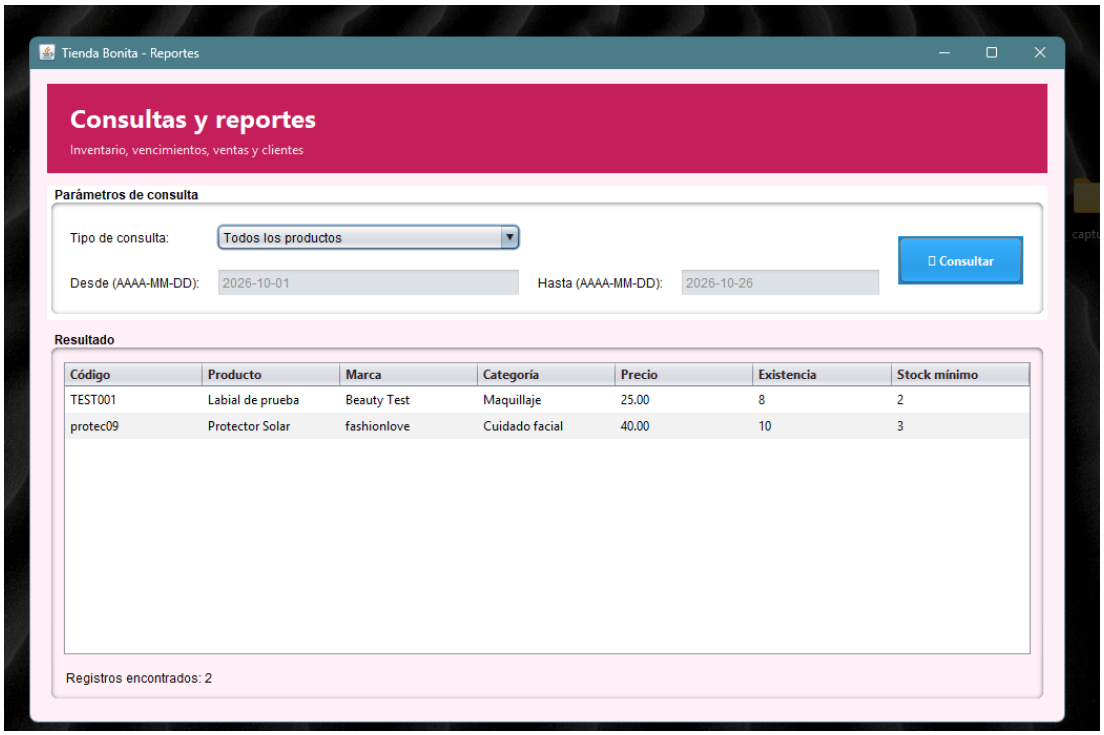
Manual de usuario — captura 14



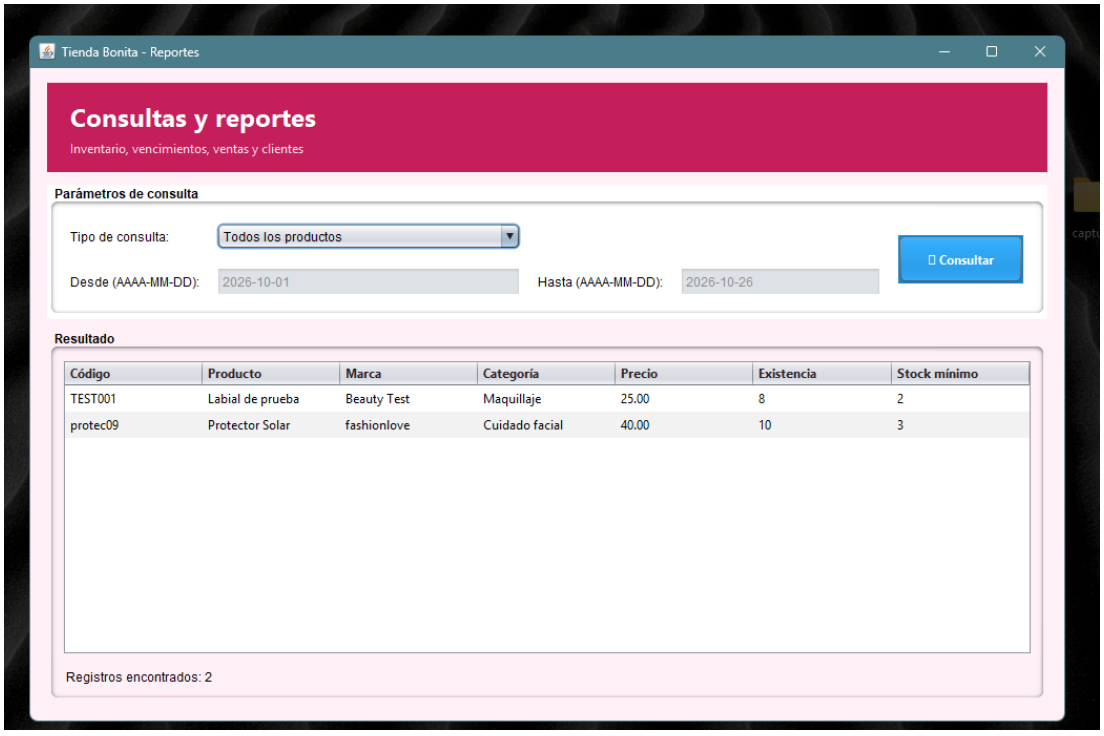
Manual de usuario — captura 15



Manual de usuario — captura 16



Manual de usuario — captura 17



Manual de usuario — captura 18



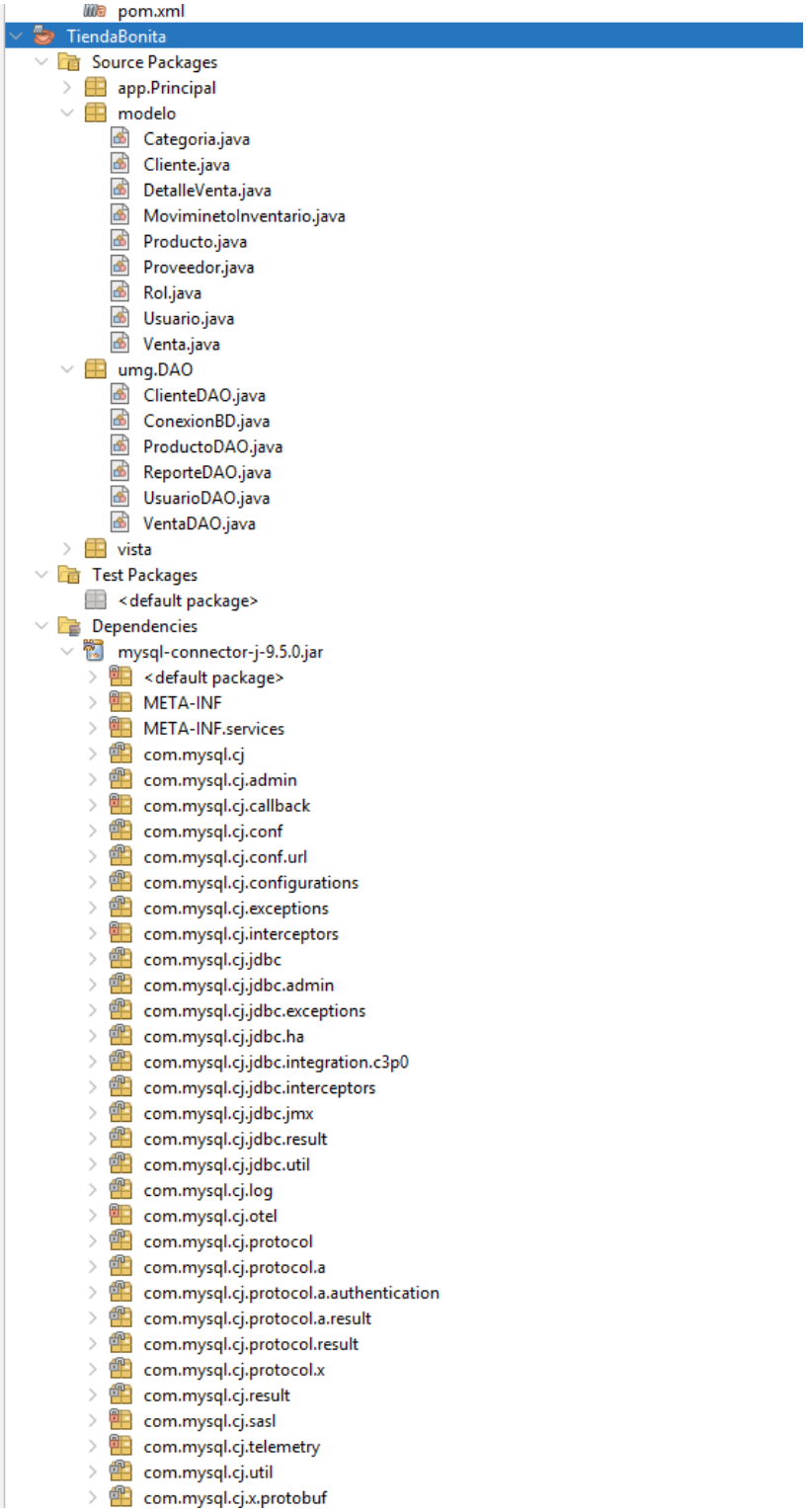
Manual de usuario — captura 19



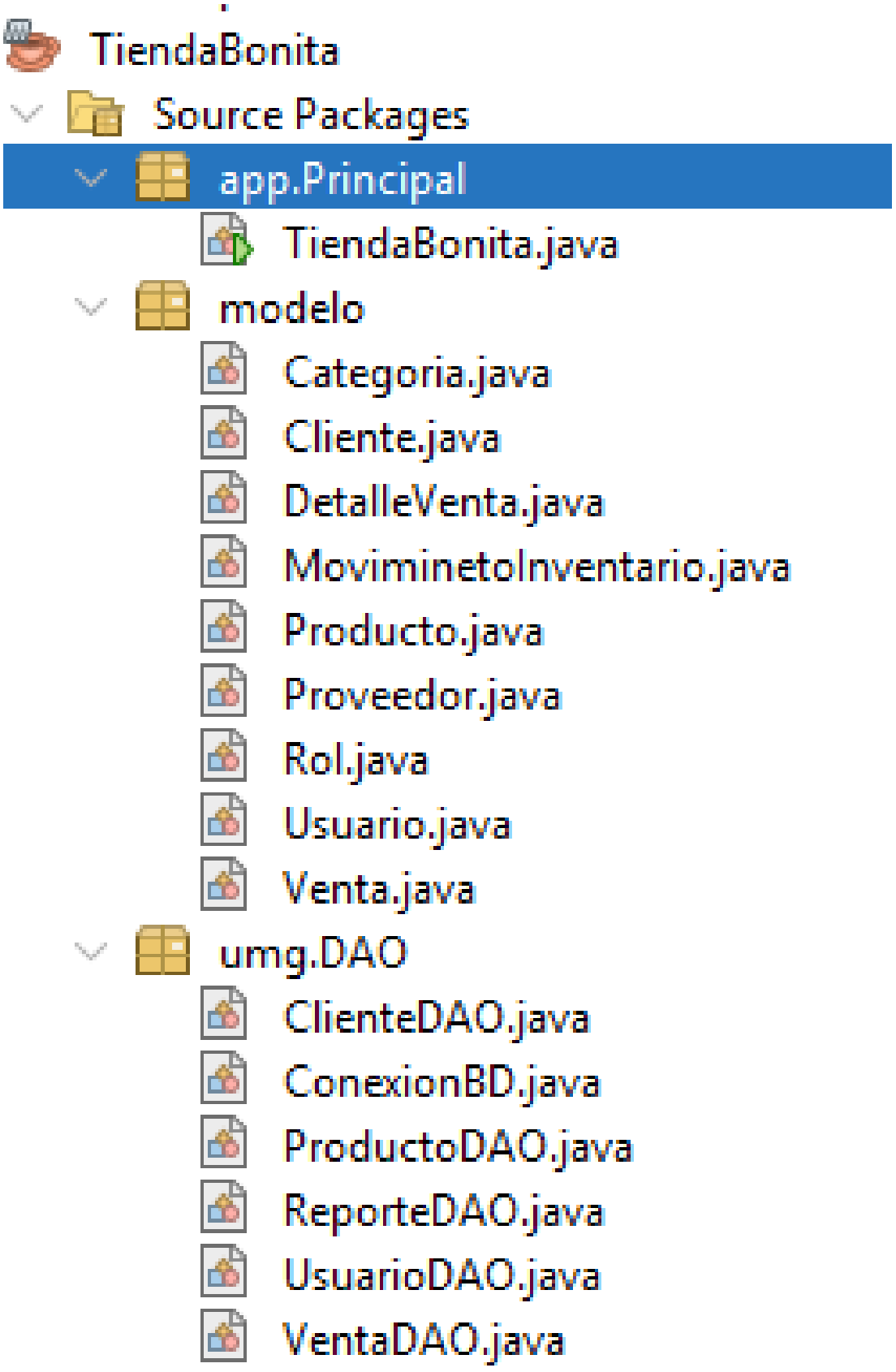
Manual de usuario — captura 20

Capturas complementarias del manual técnico

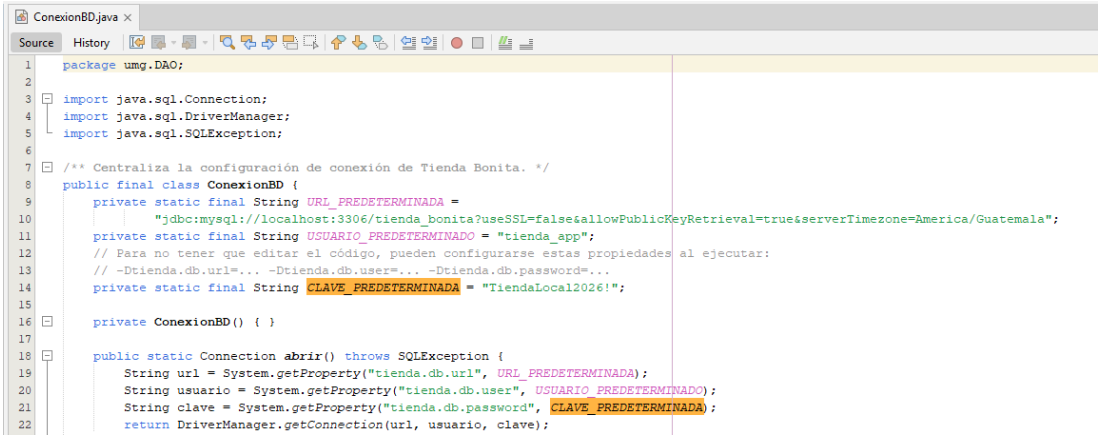
Se conservan las imágenes incrustadas en el documento original para mantener el apoyo visual y los detalles de interfaz o estructura que no aparecen en el texto extraído.



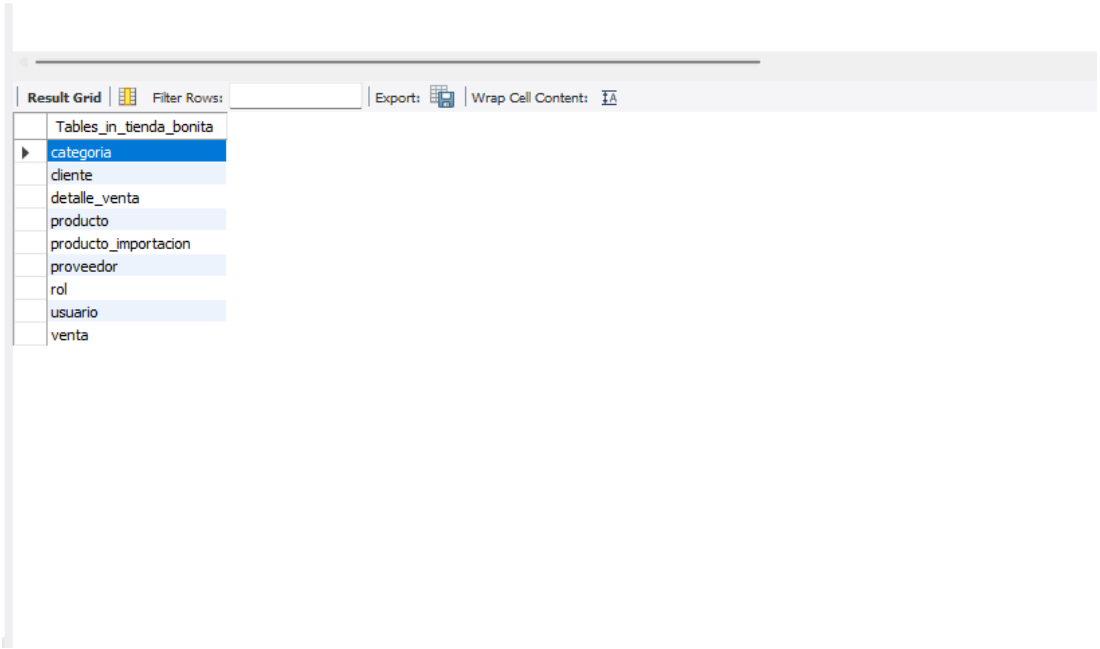
Manual técnico — captura 1



Manual técnico — captura 2



Manual técnico — captura 3



Manual técnico — captura 4



```java
package umg.DAO;

import modelo.Rol;
import modelo.Usuario;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.Locale;

/** Autenticación contra la tabla usuario; no mantiene usuarios fijos en Java. */
public class UsuarioDAO {
    public Usuario autenticar(String nombreUsuario, String contrasena) throws SQLException {
        if (nombreUsuario == null || nombreUsuario.isBlank()
                || contrasena == null || contrasena.isEmpty()) {
            return null;
        }

        String sql = "SELECT id_usuario, nombre, usuario, contrasena, estado, id_rol "
                + "FROM usuario WHERE usuario = ? LIMIT 1";

        try (Connection conexion = ConexionBD.abrir();
             PreparedStatement ps = conexion.prepareStatement(sql)) {
            ps.setString(1, nombreUsuario.trim());
            try (ResultSet rs = ps.executeQuery()) {
                if (!rs.next()) return null;

                String claveGuardada = rs.getString("contrasena");
                String estado = rs.getString("estado");
                if (claveGuardada == null || !claveGuardada.equals(contrasena)
                        || !estaActivo(estado)) {
                    return null;
                }

                int idUsuario = rs.getInt("id_usuario");
                String nombre = rs.getString("nombre");
                String usuario = rs.getString("usuario");
                int idRol = rs.getInt("id_rol");
                Rol rol = cargarRol(conexion, idRol);
                return new Usuario(idUsuario, nombre, usuario, claveGuardada, true, rol);
            }
        }
    }

    private Rol cargarRol(Connection conexion, int idRol) {
        String sql = "SELECT nombre, permisos FROM rol WHERE id_rol = ?";
        try (PreparedStatement ps = conexion.prepareStatement(sql)) {
```

Manual técnico — captura 5



```java
        try (PreparedStatement ps = conexion.prepareStatement(sql)) {
            ps.setInt(1, idRol);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return new Rol(idRol, rs.getString("nombre"), rs.getString("permisos"));
                }
            }
        } catch (SQLException ignored) {
            // La autenticación se puede completar aunque la tabla rol no tenga permisos cargados.
        }
        return new Rol(idRol, "Rol " + idRol, "");
    }

    private boolean estaActivo(String valor) {
        if (valor == null) return false;
        String normalizado = valor.trim().toLowerCase(Locale.ROOT);
        return normalizado.equals("1") || normalizado.equals("true")
                || normalizado.equals("activo") || normalizado.equals("activa")
                || normalizado.equals("si") || normalizado.equals("sí")
                || normalizado.equals("yes");
    }
}
```

Manual técnico — captura 6

```java
package vista;

import modelo.Usuario;

import javax.swing.BorderFactory;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Font;
import java.awt.GridLayout;

/** 6.2 Menú principal. */
public class FrmMenuPrincipal extends JFrame {
    private final Usuario usuario;

    public FrmMenuPrincipal(Usuario usuario) {
        super("Tienda Bonita - Menú Principal");
        this.usuario = usuario;
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(690, 470);
        setMinimumSize(new java.awt.Dimension(620, 420));
        setLocationRelativeTo(null);
        construirInterfaz();
    }

    private void construirInterfaz() {
        JPanel base = new JPanel(new BorderLayout(18, 18));
        base.setBackground(EstiloTienda.ROSA_CLARO);
        base.setBorder(BorderFactory.createEmptyBorder(20, 24, 22, 24));

        JPanel marca = new JPanel(new BorderLayout(0, 4));
        marca.setBackground(EstiloTienda.ROSA_CLARO);
        JLabel logo = new JLabel("Tienda Bonita", JLabel.CENTER);
        logo.setFont(new Font("Segoe Script", Font.BOLD, 30));
        logo.setForeground(EstiloTienda.ROSA);
        JLabel subtitulo = new JLabel("BELLEZA QUE TE ACOMPAÑA", JLabel.CENTER);
        subtitulo.setFont(new Font("Segoe UI", Font.PLAIN, 10));
        subtitulo.setForeground(EstiloTienda.ROSA);
        marca.add(logo, BorderLayout.CENTER);
        marca.add(subtitulo, BorderLayout.SOUTH);
        base.add(marca, BorderLayout.WEST);

        JPanel acciones = new JPanel(new GridLayout(5, 1, 0, 10));
        acciones.setBackground(EstiloTienda.ROSA_CLARO);
        var productos = EstiloTienda.boton("▣  PRODUCTOS / INVENTARIO", EstiloTienda.ROSA);
        var clientes = EstiloTienda.boton("  CLIENTES", EstiloTienda.AZUL);
        var ventas = EstiloTienda.boton("  VENTAS", EstiloTienda.VERDE);
        var reportes = EstiloTienda.boton("  REPORTES", EstiloTienda.NARANJA);
        var salir = EstiloTienda.boton("⇥  CERRAR SESIÓN", EstiloTienda.ROJO);
```

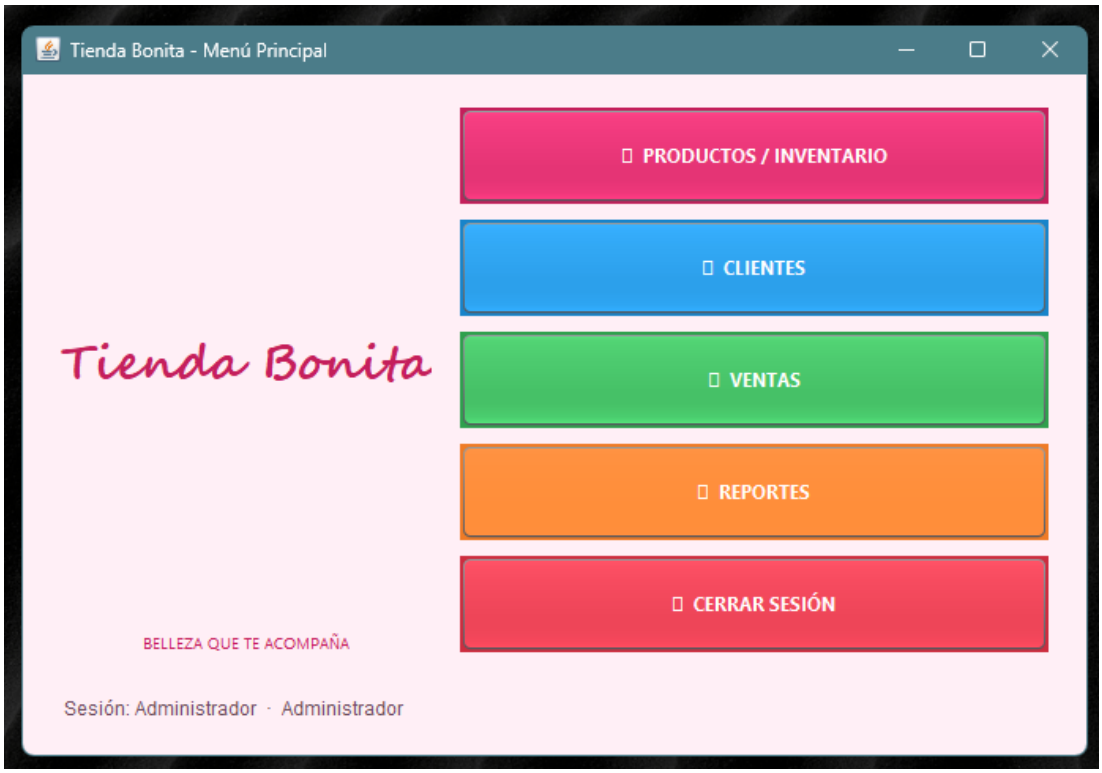
Manual técnico — captura 7

```java
        var salir = EstiloTienda.boton("⇥  CERRAR SESIÓN", EstiloTienda.ROJO);
        acciones.add(productos); acciones.add(clientes); acciones.add(ventas); acciones.add(reportes); acciones.add(salir);
        base.add(acciones, BorderLayout.CENTER);

        JLabel pie = new JLabel("Sesión: " + usuario.getNombre() + "  ·  "
                + (usuario.getRol() == null ? "Usuario" : usuario.getRol().getNombre()));
        pie.setForeground(new Color(107, 77, 91));
        pie.setBorder(BorderFactory.createEmptyBorder(8, 2, 0, 0));
        base.add(pie, BorderLayout.SOUTH);

        productos.addActionListener(e -> new FrmProductos().setVisible(true));
        clientes.addActionListener(e -> new FrmClientes().setVisible(true));
        ventas.addActionListener(e -> new FrmVentas(usuario).setVisible(true));
        reportes.addActionListener(e -> new FrmReportes().setVisible(true));
        salir.addActionListener(e -> {
            int r = JOptionPane.showConfirmDialog(this, "¿Deseas cerrar sesión?", "Cerrar sesión",
                    JOptionPane.YES_NO_OPTION, JOptionPane.QUESTION_MESSAGE);
            if (r == JOptionPane.YES_OPTION) {
                new FrmLogin().setVisible(true);
                dispose();
            }
        });
        setContentPane(base);
    }
```

Manual técnico — captura 8



Manual técnico — captura 9

```java
package umg.DAO;

import java.math.BigDecimal;
import java.sql.Connection;
import java.sql.Date;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Types;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

/** Acceso a datos para inventario/productos. */
public class ProductoDAO {
    public record CategoriaOption(int idCategoria, String nombre) {
        @Override public String toString() { return nombre; }
    }

    public record ProductoDTO(
            int idProducto,
            String codigo,
            String nombre,
            String marca,
            String calidad,
            String tonoColor,
            String presentacion,
            BigDecimal precio,
            int existencia,
            int stockMinimo,
            LocalDate fechaVencimiento,
            int idCategoria,
            String categoria,
            Integer idProveedor) {
        @Override public String toString() { return codigo + " - " + nombre; }
    }

    public List<ProductoDTO> listarProductos() throws SQLException {
        String sql = "SELECT p.id_producto, p.codigo, p.nombre, p.marca, p.calidad, "
                + "p.tono_color, p.presentacion, p.precio, p.existencia, p.stock_minimo, "
                + "p.fecha_vencimiento, p.id_categoria, COALESCE(c.nombre, 'Sin categoria') AS categoria, "
                + "p.id_proveedor FROM producto p LEFT JOIN categoria c ON c.id_categoria = p.id_categoria "
                + "ORDER BY p.nombre";
        List<ProductoDTO> productos = new ArrayList<>();
        try (Connection conexion = ConexionBD.abrir();
             PreparedStatement ps = conexion.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            while (rs.next()) productos.add(mapear(rs));
        }
        return productos;
    }
```


Manual técnico — captura 10

```java
public List<CategoriaOption> listarCategorias() throws SQLException {
    List<CategoriaOption> categorias = new ArrayList<>();
    String sql = "SELECT id_categoria, nombre FROM categoria ORDER BY nombre";
    try (Connection conexion = ConexionBD.abrir();
         PreparedStatement ps = conexion.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {
        while (rs.next()) categorias.add(new CategoriaOption(
                rs.getInt("id_categoria"), rs.getString("nombre")));
    }
    return categorias;
}

public ProductoDTO buscarProducto(String codigo) throws SQLException {
    if (codigo == null || codigo.isBlank()) return null;
    String sql = "SELECT p.id_producto, p.codigo, p.nombre, p.marca, p.calidad, "
            + "p.tono_color, p.presentacion, p.precio, p.existencia, p.stock_minimo, "
            + "p.fecha_vencimiento, p.id_categoria, COALESCE(c.nombre, 'Sin categoria') AS categoria, "
            + "p.id_proveedor FROM producto p LEFT JOIN categoria c ON c.id_categoria = p.id_categoria "
            + "WHERE p.codigo = ? LIMIT 1";
    try (Connection conexion = ConexionBD.abrir();
         PreparedStatement ps = conexion.prepareStatement(sql)) {
        ps.setString(1, codigo.trim());
        try (ResultSet rs = ps.executeQuery()) {
            return rs.next() ? mapear(rs) : null;
        }
    }
}

public boolean guardarProducto(String codigo, String nombre, String marca,
        String calidad, String tonoColor, String presentacion,
        double precio, int existencia, int stockMinimo,
        Integer idCategoria, Integer idProveedor) throws SQLException {
    validar(codigo, nombre, precio, existencia, stockMinimo, idCategoria);
    String sql = "INSERT INTO producto (codigo, nombre, marca, calidad, tono_color, "
            + "presentacion, precio, existencia, stock_minimo, id_categoria, id_proveedor) "
            + "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
    try (Connection conexion = ConexionBD.abrir();
         PreparedStatement ps = conexion.prepareStatement(sql)) {
        ps.setString(1, codigo.trim());
        ps.setString(2, nombre.trim());
        ps.setString(3, texto(marca));
        ps.setString(4, texto(calidad));
        ps.setString(5, texto(tonoColor));
        ps.setString(6, texto(presentacion));
        ps.setBigDecimal(7, BigDecimal.valueOf(precio));
        ps.setInt(8, existencia);
        ps.setInt(9, stockMinimo);
        ps.setInt(10, idCategoria);
        setNullableInt(ps, 11, idProveedor);
        return ps.executeUpdate() == 1;
    }
}
```

Manual técnico — captura 11

```java
}

public boolean actualizarProducto(String codigo, String nombre, String marca,
        String calidad, String tonoColor, String presentacion,
        double precio, int existencia, int stockMinimo,
        Integer idCategoria, Integer idProveedor) throws SQLException {
    validar(codigo, nombre, precio, existencia, stockMinimo, idCategoria);
    String sql = "UPDATE producto SET nombre = ?, marca = ?, calidad = ?, tono_color = ?, "
            + "presentacion = ?, precio = ?, existencia = ?, stock_minimo = ?, id_categoria = ?, "
            + "id_proveedor = COALESCE(?, id_proveedor) WHERE codigo = ?";
    try (Connection conexion = ConexionBD.abrir();
         PreparedStatement ps = conexion.prepareStatement(sql)) {
        ps.setString(1, nombre.trim());
        ps.setString(2, texto(marca));
        ps.setString(3, texto(calidad));
        ps.setString(4, texto(tonoColor));
        ps.setString(5, texto(presentacion));
        ps.setBigDecimal(6, BigDecimal.valueOf(precio));
        ps.setInt(7, existencia);
        ps.setInt(8, stockMinimo);
        ps.setInt(9, idCategoria);
        setNullableInt(ps, 10, idProveedor);
        ps.setString(11, codigo.trim());
        return ps.executeUpdate() > 0;
    }
}

public boolean eliminarProducto(String codigo) throws SQLException {
    String sql = "DELETE FROM producto WHERE codigo = ?";
    try (Connection conexion = ConexionBD.abrir();
         PreparedStatement ps = conexion.prepareStatement(sql)) {
        ps.setString(1, codigo.trim());
        return ps.executeUpdate() > 0;
    }
}

public boolean actualizarExistencia(String codigo, int nuevaExistencia) throws SQLException {
    if (nuevaExistencia < 0) throw new IllegalArgumentException("La existencia no puede ser negativa.");
    String sql = "UPDATE producto SET existencia = ? WHERE codigo = ?";
    try (Connection conexion = ConexionBD.abrir();
         PreparedStatement ps = conexion.prepareStatement(sql)) {
        ps.setInt(1, nuevaExistencia);
        ps.setString(2, codigo.trim());
        return ps.executeUpdate() > 0;
    }
}

/** Conserva la salida de consola útil para pruebas manuales. */
public void imprimirProductos() {
    try {
        System.out.println("PRODUCTOS DE TIENDA BONITA");
```

Manual técnico — captura 12

```
            System.out.println("PRODUCTOS DE TIENDA BONITA");
            for (ProductoDTO p : listarProductos()) {
                System.out.printf("%s | %s | %s | Q%s | Existencia: %d%n",
                        p.codigo(), p.nombre(), p.marca(), p.precio(), p.existencia());
            }
        } catch (SQLException e) {
            System.err.println("Error al leer productos: " + e.getMessage());
        }
    }

    private ProductoDTO mapear(ResultSet rs) throws SQLException {
        int idProveedorLeido = rs.getInt("id_proveedor");
        Integer idProveedor = rs.wasNull() ? null : idProveedorLeido;
        return new ProductoDTO(
                rs.getInt("id_producto"), rs.getString("codigo"), rs.getString("nombre"),
                rs.getString("marca"), rs.getString("calidad"), rs.getString("tono_color"),
                rs.getString("presentacion"), rs.getBigDecimal("precio"),
                rs.getInt("existencia"), rs.getInt("stock_minimo"), leerFecha(rs),
                rs.getInt("id_categoria"), rs.getString("categoria"), idProveedor);
    }

    private LocalDate leerFecha(ResultSet rs) throws SQLException {
        Date fecha = rs.getDate("fecha_vencimiento");
        return fecha == null ? null : fecha.toLocalDate();
    }

    private void validar(String codigo, String nombre, double precio, int existencia,
            int stockMinimo, Integer idCategoria) {
        if (codigo == null || codigo.isBlank()) throw new IllegalArgumentException("El código es obligatorio.");
        if (nombre == null || nombre.isBlank()) throw new IllegalArgumentException("El nombre es obligatorio.");
        if (!Double.isFinite(precio) || precio <= 0) throw new IllegalArgumentException("El precio debe ser mayor que cero.");
        if (existencia < 0) throw new IllegalArgumentException("La existencia no puede ser negativa.");
        if (stockMinimo < 0) throw new IllegalArgumentException("El stock mínimo no puede ser negativo.");
        if (idCategoria == null || idCategoria <= 0) throw new IllegalArgumentException("Selecciona una categoría.");
    }

    private String texto(String valor) { return valor == null ? "" : valor.trim(); }

    private void setNullableInt(PreparedStatement ps, int indice, Integer valor) throws SQLException {
        if (valor == null) ps.setNull(indice, Types.INTEGER);
        else ps.setInt(indice, valor);
    }
}
```

Manual técnico — captura 13

```
package umg.DAO;

import modelo.Cliente;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

/** CRUD para clientes. */
public class ClienteDAO {
    public List<Cliente> listarClientes() throws SQLException {
        String sql = "SELECT id_cliente, nombre, nit, telefono, correo, direccion FROM cliente ORDER BY nombre";
        List<Cliente> clientes = new ArrayList<>();
        try (Connection c = ConexionBD.abrir(); PreparedStatement ps = c.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            while (rs.next()) clientes.add(mapear(rs));
        }
        return clientes;
    }

    public List<Cliente> buscarClientes(String criterio) throws SQLException {
        String sql = "SELECT id_cliente, nombre, nit, telefono, correo, direccion FROM cliente "
                + "WHERE nombre LIKE ? OR nit LIKE ? ORDER BY nombre";
        List<Cliente> clientes = new ArrayList<>();
        String filtro = "%" + (criterio == null ? "" : criterio.trim()) + "%";
        try (Connection c = ConexionBD.abrir(); PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setString(1, filtro);
            ps.setString(2, filtro);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) clientes.add(mapear(rs));
            }
        }
        return clientes;
    }

    public boolean guardarCliente(Cliente cliente) throws SQLException {
        validar(cliente);
        String sql = "INSERT INTO cliente (nombre, nit, telefono, correo, direccion) VALUES (?, ?, ?, ?, ?)";
        try (Connection c = ConexionBD.abrir(); PreparedStatement ps = c.prepareStatement(sql)) {
            asignarCampos(ps, cliente, false);
            return ps.executeUpdate() == 1;
        }
    }

    public boolean actualizarCliente(Cliente cliente) throws SQLException {
        validar(cliente);
        if (cliente.getIdCliente() <= 0) throw new IllegalArgumentException("Selecciona un cliente de la tabla.");
        String sql = "UPDATE cliente SET nombre = ?, nit = ?, telefono = ?, correo = ?, direccion = ? WHERE id_cliente = ?";
        try (Connection c = ConexionBD.abrir(); PreparedStatement ps = c.prepareStatement(sql)) {
```

Manual técnico — captura 14

```
        try (Connection c = ConexionBD.abrir(); PreparedStatement ps = c.prepareStatement(sql)) {
            asignarCampos(ps, cliente, true);
            ps.setInt(6, cliente.getIdCliente());
            return ps.executeUpdate() > 0;
        }
    }

    public boolean eliminarCliente(int idCliente) throws SQLException {
        String sql = "DELETE FROM cliente WHERE id_cliente = ?";
        try (Connection c = ConexionBD.abrir(); PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setInt(1, idCliente);
            return ps.executeUpdate() > 0;
        }
    }

    private Cliente mapear(ResultSet rs) throws SQLException {
        return new Cliente(rs.getInt("id_cliente"), rs.getString("nombre"),
                rs.getString("nit"), rs.getString("telefono"), rs.getString("correo"),
                rs.getString("direccion"));
    }

    private void asignarCampos(PreparedStatement ps, Cliente cliente, boolean actualizacion) throws SQLException {
        ps.setString(1, normalizar(cliente.getNombre()));
        ps.setString(2, normalizar(cliente.getNit()));
        ps.setString(3, normalizar(cliente.getTelefono()));
        ps.setString(4, normalizar(cliente.getCorreo()));
        ps.setString(5, normalizar(cliente.getDireccion()));
    }

    private void validar(Cliente cliente) {
        if (cliente == null || cliente.getNombre() == null || cliente.getNombre().isBlank()) {
            throw new IllegalArgumentException("El nombre del cliente es obligatorio.");
        }
    }

    private String normalizar(String s) { return s == null ? "" : s.trim(); }
```

Manual técnico — captura 15

```
package umg.DAO;

import java.math.BigDecimal;
import java.sql.Connection;
import java.sql.Date;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/** Registra venta y detalle en una sola transacción y descuenta inventario en MySQL. */
public class VentaDAO {
    public record DetalleVentaInput(int idProducto, int cantidad) { }
    public record VentaRegistrada(int idVenta, BigDecimal total) { }
    private record ProductoBloqueado(String codigo, String nombre, BigDecimal precio,
                                     int existencia, LocalDate fechaVencimiento) { }

    public VentaRegistrada confirmarVenta(int idCliente, int idUsuario, LocalDate fecha,
                                          List<DetalleVentaInput> detalles) throws SQLException {
        if (idCliente <= 0) throw new IllegalArgumentException("Selecciona un cliente.");
        if (idUsuario <= 0) throw new IllegalArgumentException("No se pudo identificar al vendedor. Cierra sesión e ingresa nuevamente.");
        if (fecha == null) throw new IllegalArgumentException("La fecha de venta es obligatoria.");
        if (detalles == null || detalles.isEmpty()) throw new IllegalArgumentException("Agrega al menos un producto a la venta.");
        for (DetalleVentaInput d : detalles) {
            if (d.idProducto() <= 0 || d.cantidad() <= 0) throw new IllegalArgumentException("Todas las cantidades deben ser mayores que cero.");
        }

        try (Connection c = ConexionBD.abrir()) {
            boolean autoCommitAnterior = c.getAutoCommit();
            c.setAutoCommit(false);
            try {
                Map<Integer, ProductoBloqueado> productos = new HashMap<>();
                BigDecimal total = BigDecimal.ZERO;
                for (DetalleVentaInput d : detalles) {
                    ProductoBloqueado p = productos.get(d.idProducto());
                    if (p == null) {
                        p = bloquearProducto(c, d.idProducto());
                        productos.put(d.idProducto(), p);
                    }
                    int solicitadoAcumulado = detalles.stream()
                            .filter(x -> x.idProducto() == d.idProducto())
                            .mapToInt(DetalleVentaInput::cantidad).sum();
                    if (p.existencia() < solicitadoAcumulado) {
                        throw new SQLException("Existencia insuficiente para " + p.nombre()
                                + ". Disponible: " + p.existencia() + ".");
                    }
                    if (p.fechaVencimiento() != null && p.fechaVencimiento().isBefore(fecha)) {
                        throw new SQLException("No se puede vender el producto vencido: " + p.nombre() + ".");
```

Manual técnico — captura 16

```
                }
            }
            for (DetalleVentaInput d : detalles) {
                ProductoBloqueado p = productos.get(d.idProducto());
                total = total.add(p.precio().multiply(BigDecimal.valueOf(d.cantidad())));
            }

            int idVenta;
            String sqlVenta = "INSERT INTO venta (fecha, id_cliente, id_usuario, total, estado) VALUES (?, ?, ?, ?, ?)";
            try (PreparedStatement ps = c.prepareStatement(sqlVenta, PreparedStatement.RETURN_GENERATED_KEYS)) {
                ps.setDate(1, Date.valueOf(fecha));
                ps.setInt(2, idCliente);
                ps.setInt(3, idUsuario);
                ps.setBigDecimal(4, total);
                ps.setString(5, "Confirmada");
                ps.executeUpdate();
                try (ResultSet keys = ps.getGeneratedKeys()) {
                    if (!keys.next()) throw new SQLException("MySQL no devolvió el número de venta.");
                    idVenta = keys.getInt(1);
                }
            }

            String sqlDetalle = "INSERT INTO detalle_venta (id_venta, id_producto, cantidad, precio, subtotal) VALUES (?, ?, ?, ?, ?)";
            String sqlStock = "UPDATE producto SET existencia = existencia - ? WHERE id_producto = ? AND existencia >= ?";
            try (PreparedStatement psDetalle = c.prepareStatement(sqlDetalle);
                 PreparedStatement psStock = c.prepareStatement(sqlStock)) {
                for (DetalleVentaInput d : detalles) {
                    ProductoBloqueado p = productos.get(d.idProducto());
                    BigDecimal subtotal = p.precio().multiply(BigDecimal.valueOf(d.cantidad()));
                    psDetalle.setInt(1, idVenta);
                    psDetalle.setInt(2, d.idProducto());
                    psDetalle.setInt(3, d.cantidad());
                    psDetalle.setBigDecimal(4, p.precio());
                    psDetalle.setBigDecimal(5, subtotal);
                    psDetalle.executeUpdate();

                    psStock.setInt(1, d.cantidad());
                    psStock.setInt(2, d.idProducto());
                    psStock.setInt(3, d.cantidad());
                    if (psStock.executeUpdate() != 1) {
                        throw new SQLException("No se pudo actualizar el inventario de " + p.nombre() + ". Intenta nuevamente.");
                    }
                }
            }
            c.commit();
            return new VentaRegistrada(idVenta, total);
        } catch (SQLException | RuntimeException e) {
            try { c.rollback(); } catch (SQLException rollbackError) { e.addSuppressed(rollbackError); }
            throw e;
        } finally {
            try { c.setAutoCommit(autoCommitAnterior); } catch (SQLException ignored) { }
        }
```

:LAVE_PREDETERMINADA Previous Next Select 2 matc

Manual técnico — captura 17

```
            }
        }
    }

    private ProductoBloqueado bloquearProducto(Connection c, int idProducto) throws SQLException {
        String sql = "SELECT codigo, nombre, precio, existencia, fecha_vencimiento FROM producto WHERE id_producto = ? FOR UPDATE";
        try (PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setInt(1, idProducto);
            try (ResultSet rs = ps.executeQuery()) {
                if (!rs.next()) throw new SQLException("Uno de los productos seleccionados ya no existe.");
                Date fecha = rs.getDate("fecha_vencimiento");
                return new ProductoBloqueado(rs.getString("codigo"), rs.getString("nombre"),
                        rs.getBigDecimal("precio"), rs.getInt("existencia"),
                        fecha == null ? null : fecha.toLocalDate());
            }
        }
    }
```

Manual técnico — captura 18

```java
package umg.DAO;

import java.sql.Connection;
import java.sql.Date;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.SQLException;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

/** Consultas disponibles en el módulo de reportes. */
public class ReporteDAO {
    public record Resultado(String[] columnas, List<Object[]> filas) { }

    public Resultado consultar(String tipo, LocalDate desde, LocalDate hasta) throws SQLException {
        String sql;
        boolean usaFechas = false;
        switch (tipo) {
            case "Todos los productos" -> sql = "SELECT p.codigo AS Código, p.nombre AS Producto, p.marca AS Marca, "
                    + "c.nombre AS Categoría, p.precio AS Precio, p.existencia AS Existencia, p.stock_minimo AS `Stock mínimo` "
                    + "FROM producto p LEFT JOIN categoria c ON c.id_categoria = p.id_categoria ORDER BY p.nombre";
            case "Productos con stock bajo" -> sql = "SELECT p.codigo AS Código, p.nombre AS Producto, p.existencia AS Existencia, "
                    + "p.stock_minimo AS `Stock mínimo`, c.nombre AS Categoría FROM producto p "
                    + "LEFT JOIN categoria c ON c.id_categoria = p.id_categoria WHERE p.existencia <= p.stock_minimo ORDER BY p.existencia";
            case "Productos por categoría" -> sql = "SELECT c.nombre AS Categoría, p.codigo AS Código, p.nombre AS Producto, "
                    + "p.precio AS Precio, p.existencia AS Existencia FROM producto p "
                    + "LEFT JOIN categoria c ON c.id_categoria = p.id_categoria ORDER BY c.nombre, p.nombre";
            case "Productos vencidos" -> sql = "SELECT p.codigo AS Código, p.nombre AS Producto, p.fecha_vencimiento AS `Fecha de vencimiento`, "
                    + "p.existencia AS Existencia FROM producto p WHERE p.fecha_vencimiento IS NOT NULL "
                    + "AND p.fecha_vencimiento < CURDATE() ORDER BY p.fecha_vencimiento";
            case "Ventas realizadas" -> sql = "SELECT v.id_venta AS `No. venta`, v.fecha AS Fecha, c.nombre AS Cliente, "
                    + "u.nombre AS Vendedor, v.total AS Total, v.estado AS Estado FROM venta v "
                    + "LEFT JOIN cliente c ON c.id_cliente = v.id_cliente "
                    + "LEFT JOIN usuario u ON u.id_usuario = v.id_usuario ORDER BY v.fecha DESC, v.id_venta DESC";
            case "Ventas por fecha" -> {
                sql = "SELECT v.id_venta AS `No. venta`, v.fecha AS Fecha, c.nombre AS Cliente, "
                        + "u.nombre AS Vendedor, v.total AS Total, v.estado AS Estado FROM venta v "
                        + "LEFT JOIN cliente c ON c.id_cliente = v.id_cliente "
                        + "LEFT JOIN usuario u ON u.id_usuario = v.id_usuario "
                        + "WHERE v.fecha BETWEEN ? AND ? ORDER BY v.fecha DESC, v.id_venta DESC";
                usaFechas = true;
            }
            case "Clientes registrados" -> sql = "SELECT id_cliente AS ID, nombre AS Nombre, nit AS NIT, "
                    + "telefono AS Teléfono, correo AS Correo, direccion AS Dirección FROM cliente ORDER BY nombre";
            default -> throw new IllegalArgumentException("Selecciona un tipo de reporte válido.");
        }

        try (Connection c = ConexionBD.abrir(); PreparedStatement ps = c.prepareStatement(sql)) {
            if (usaFechas) {
                LocalDate inicio = desde == null ? LocalDate.now().withDayOfMonth(1) : desde;
                LocalDate fin = hasta == null ? LocalDate.now() : hasta;
                if (inicio.isAfter(fin)) throw new IllegalArgumentException("La fecha inicial no puede ser posterior a la fecha final.");
                ps.setDate(1, Date.valueOf(inicio));
                ps.setDate(2, Date.valueOf(fin));
            }
            try (ResultSet rs = ps.executeQuery()) {
                ResultSetMetaData md = rs.getMetaData();
                int cantidadColumnas = md.getColumnCount();
                String[] columnas = new String[cantidadColumnas];
                for (int i = 0; i < cantidadColumnas; i++) columnas[i] = md.getColumnLabel(i + 1);
                List<Object[]> filas = new ArrayList<>();
                while (rs.next()) {
                    Object[] fila = new Object[cantidadColumnas];
                    for (int i = 0; i < cantidadColumnas; i++) fila[i] = rs.getObject(i + 1);
                    filas.add(fila);
                }
                return new Resultado(columnas, filas);
            }
        }
    }
}
```

Manual técnico — captura 19

Manual técnico — captura 20

```java
package umg.DAO;

import java.math.BigDecimal;
import java.sql.Connection;
import java.sql.Date;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/** Registra venta y detalle en una sola transacción y descuenta inventario en MySQL. */
public class VentaDAO {
    public record DetalleVentaInput(int idProducto, int cantidad) { }
    public record VentaRegistrada(int idVenta, BigDecimal total) { }
    private record ProductoBloqueado(String codigo, String nombre, BigDecimal precio,
                                     int existencia, LocalDate fechaVencimiento) { }

    public VentaRegistrada confirmarVenta(int idCliente, int idUsuario, LocalDate fecha,
                                          List<DetalleVentaInput> detalles) throws SQLException {
        if (idCliente <= 0) throw new IllegalArgumentException("Selecciona un cliente.");
        if (idUsuario <= 0) throw new IllegalArgumentException("No se pudo identificar al vendedor. Cierra sesión e ingresa nuevamente.");
        if (fecha == null) throw new IllegalArgumentException("La fecha de venta es obligatoria.");
        if (detalles == null || detalles.isEmpty()) throw new IllegalArgumentException("Agrega al menos un producto a la venta.");
        for (DetalleVentaInput d : detalles) {
            if (d.idProducto() <= 0 || d.cantidad() <= 0) throw new IllegalArgumentException("Todas las cantidades deben ser mayores que cero.");
        }

        try (Connection c = ConexionBD.abrir()) {
            boolean autoCommitAnterior = c.getAutoCommit();
            c.setAutoCommit(false);
            try {
                Map<Integer, ProductoBloqueado> productos = new HashMap<>();
                BigDecimal total = BigDecimal.ZERO;
                for (DetalleVentaInput d : detalles) {
                    ProductoBloqueado p = productos.get(d.idProducto());
                    if (p == null) {
                        p = bloquearProducto(c, d.idProducto());
                        productos.put(d.idProducto(), p);
                    }
                    int solicitadoAcumulado = detalles.stream()
                            .filter(x -> x.idProducto() == d.idProducto())
                            .mapToInt(DetalleVentaInput::cantidad).sum();
                    if (p.existencia() < solicitadoAcumulado) {
                        throw new SQLException("Existencia insuficiente para " + p.nombre()
                                + ". Disponible: " + p.existencia() + ".");
```

Manual técnico — captura 21

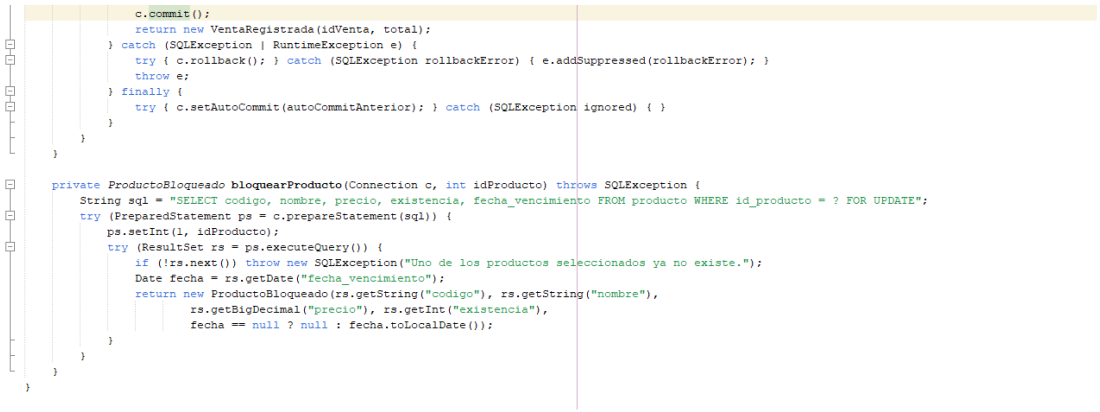
```java
                    }
                    if (p.fechaVencimiento() != null && p.fechaVencimiento().isBefore(fecha)) {
                        throw new SQLException("No se puede vender el producto vencido: " + p.nombre() + ".");
                    }
                }
                for (DetalleVentaInput d : detalles) {
                    ProductoBloqueado p = productos.get(d.idProducto());
                    total = total.add(p.precio().multiply(BigDecimal.valueOf(d.cantidad())));
                }

                int idVenta;
                String sqlVenta = "INSERT INTO venta (fecha, id_cliente, id_usuario, total, estado) VALUES (?, ?, ?, ?, ?)";
                try (PreparedStatement ps = c.prepareStatement(sqlVenta, PreparedStatement.RETURN_GENERATED_KEYS)) {
                    ps.setDate(1, Date.valueOf(fecha));
                    ps.setInt(2, idCliente);
                    ps.setInt(3, idUsuario);
                    ps.setBigDecimal(4, total);
                    ps.setString(5, "Confirmada");
                    ps.executeUpdate();
                    try (ResultSet keys = ps.getGeneratedKeys()) {
                        if (!keys.next()) throw new SQLException("MySQL no devolvió el número de venta.");
                        idVenta = keys.getInt(1);
                    }
                }

                String sqlDetalle = "INSERT INTO detalle_venta (id_venta, id_producto, cantidad, precio, subtotal) VALUES (?, ?, ?, ?, ?)";
                String sqlStock = "UPDATE producto SET existencia = existencia - ? WHERE id_producto = ? AND existencia >= ?";
                try (PreparedStatement psDetalle = c.prepareStatement(sqlDetalle);
                     PreparedStatement psStock = c.prepareStatement(sqlStock)) {
                    for (DetalleVentaInput d : detalles) {
                        ProductoBloqueado p = productos.get(d.idProducto());
                        BigDecimal subtotal = p.precio().multiply(BigDecimal.valueOf(d.cantidad()));
                        psDetalle.setInt(1, idVenta);
                        psDetalle.setInt(2, d.idProducto());
                        psDetalle.setInt(3, d.cantidad());
                        psDetalle.setBigDecimal(4, p.precio());
                        psDetalle.setBigDecimal(5, subtotal);
                        psDetalle.executeUpdate();

                        psStock.setInt(1, d.cantidad());
                        psStock.setInt(2, d.idProducto());
                        psStock.setInt(3, d.cantidad());
                        if (psStock.executeUpdate() != 1) {
                            throw new SQLException("No se pudo actualizar el inventario de " + p.nombre() + ". Intenta nuevamente.");
                        }
                    }
                }
```

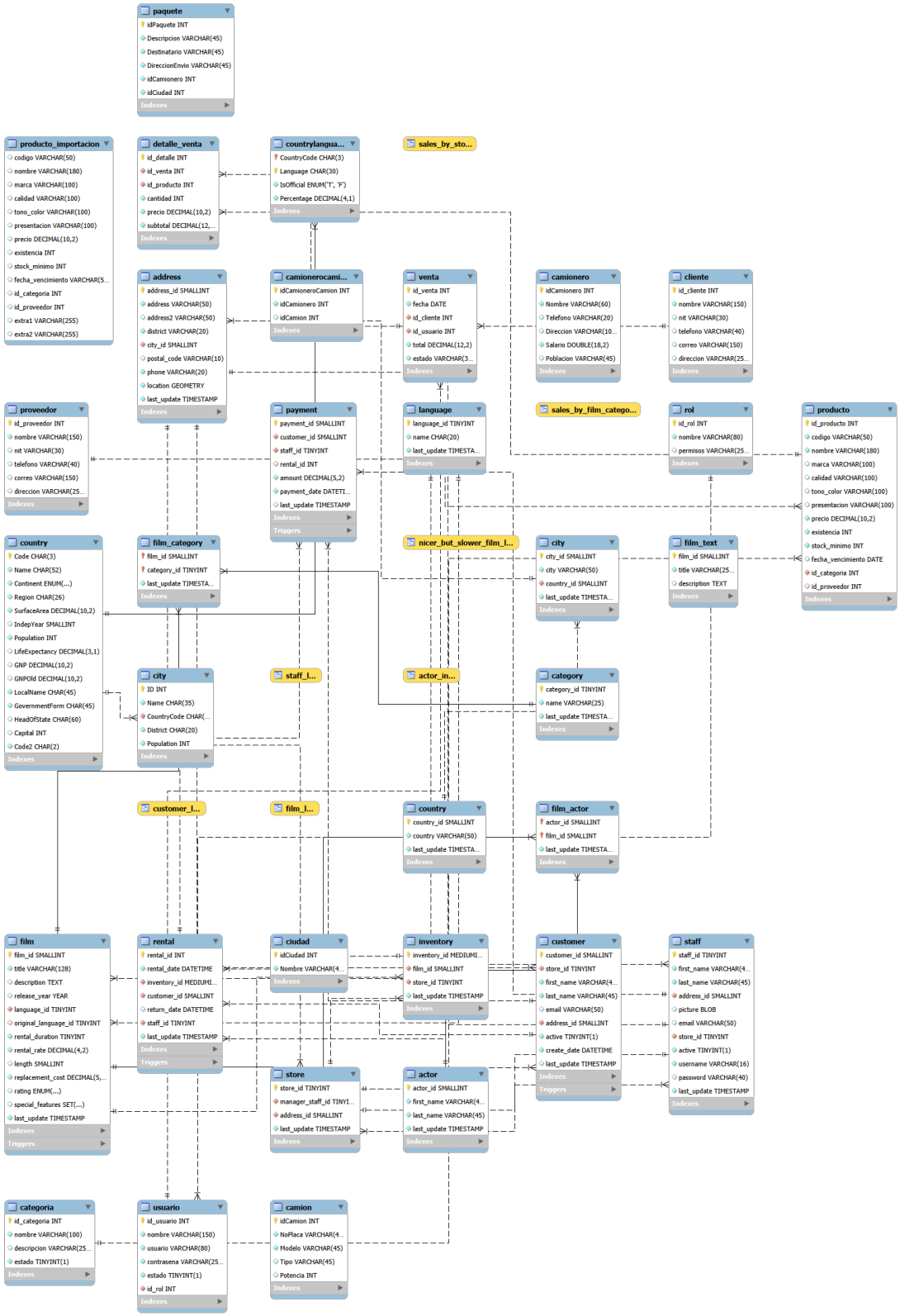
Manual técnico — captura 22



```
                c.commit();
                return new VentaRegistrada(idVenta, total);
            } catch (SQLException | RuntimeException e) {
                try { c.rollback(); } catch (SQLException rollbackError) { e.addSuppressed(rollbackError); }
                throw e;
            } finally {
                try { c.setAutoCommit(autoCommitAnterior); } catch (SQLException ignored) { }
            }
        }
    }

    private ProductoBloqueado bloquearProducto(Connection c, int idProducto) throws SQLException {
        String sql = "SELECT codigo, nombre, precio, existencia, fecha_vencimiento FROM producto WHERE id_producto = ? FOR UPDATE";
        try (PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setInt(1, idProducto);
            try (ResultSet rs = ps.executeQuery()) {
                if (!rs.next()) throw new SQLException("Uno de los productos seleccionados ya no existe.");
                Date fecha = rs.getDate("fecha_vencimiento");
                return new ProductoBloqueado(rs.getString("codigo"), rs.getString("nombre"),
                        rs.getBigDecimal("precio"), rs.getInt("existencia"),
                        fecha == null ? null : fecha.toLocalDate());
            }
        }
    }
}
```

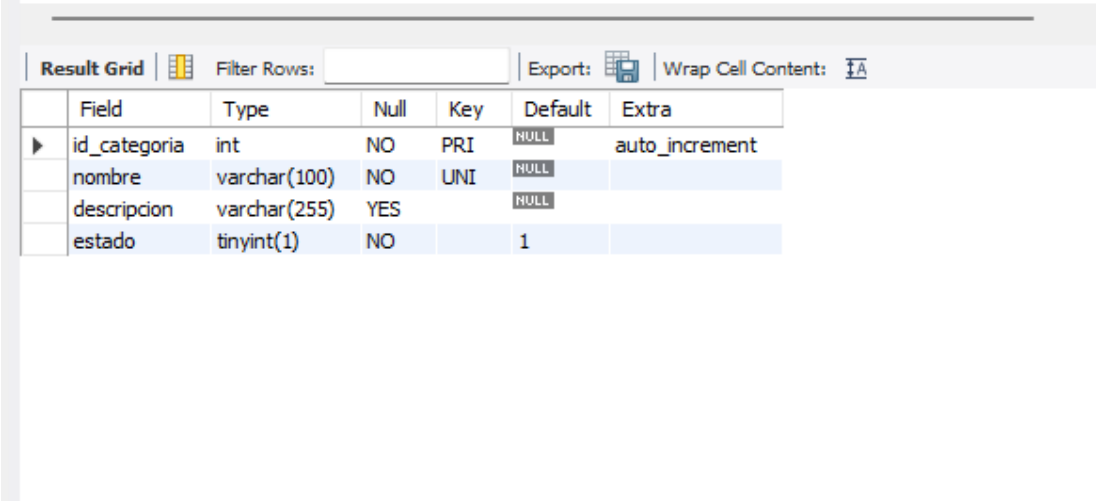
Manual técnico — captura 23



Manual técnico — captura 24

Capturas complementarias del diccionario de datos

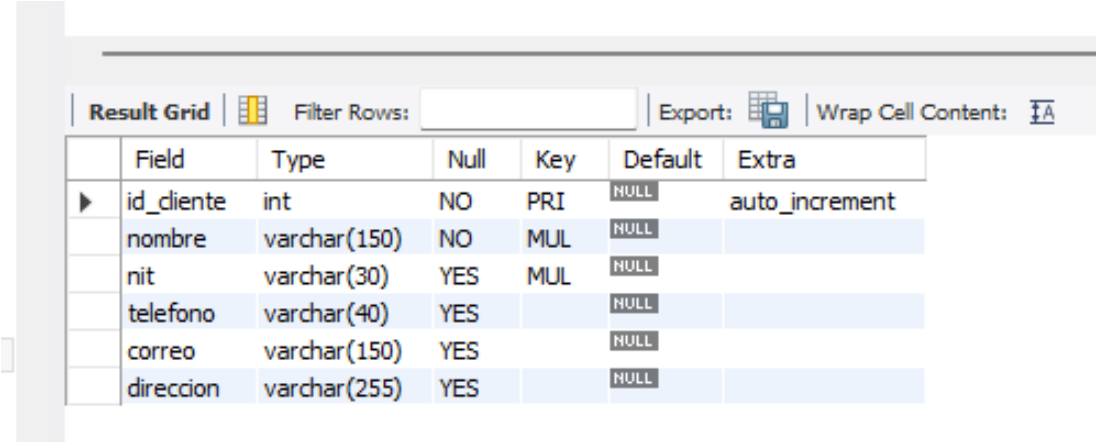
Se conservan las imágenes incrustadas en el documento original para mantener el apoyo visual y los detalles de interfaz o estructura que no aparecen en el texto extraído.



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id_categoria	int	NO	PRI	NULL	auto_increment
nombre	varchar(100)	NO	UNI	NULL	
descripcion	varchar(255)	YES		NULL	
estado	tinyint(1)	NO		1	

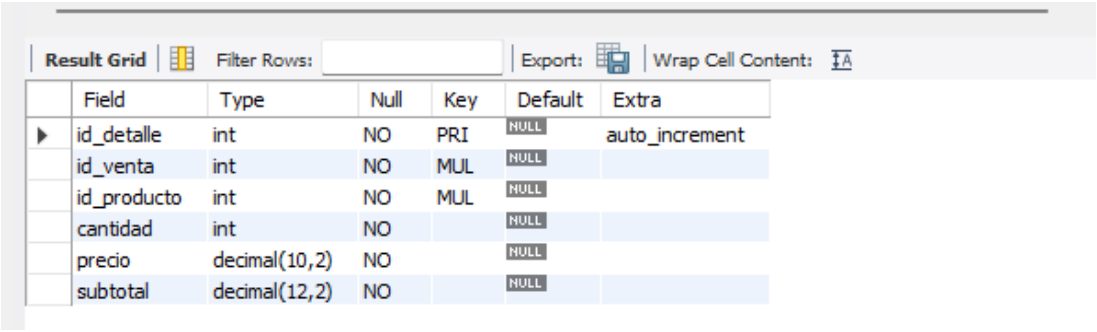
Diccionario de datos — captura 1



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id_cliente	int	NO	PRI	NULL	auto_increment
nombre	varchar(150)	NO	MUL	NULL	
nit	varchar(30)	YES	MUL	NULL	
telefono	varchar(40)	YES		NULL	
correo	varchar(150)	YES		NULL	
direccion	varchar(255)	YES		NULL	

Diccionario de datos — captura 2



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id_detalle	int	NO	PRI	NULL	auto_increment
id_venta	int	NO	MUL	NULL	
id_producto	int	NO	MUL	NULL	
cantidad	int	NO		NULL	
precio	decimal(10,2)	NO		NULL	
subtotal	decimal(12,2)	NO		NULL	

Diccionario de datos — captura 3

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id_producto	int	NO	PRI	NULL	auto_increment
codigo	varchar(50)	NO	UNI	NULL	
nombre	varchar(180)	NO	MUL	NULL	
marca	varchar(100)	YES		NULL	
calidad	varchar(100)	YES		NULL	
tono_color	varchar(100)	YES		NULL	
presentacion	varchar(100)	YES		NULL	
precio	decimal(10,2)	NO		0.00	
existencia	int	NO		0	
stock_minimo	int	NO		0	
fecha_venci...	date	YES	MUL	NULL	
id_categoria	int	NO	MUL	NULL	
id_proveedor	int	YES	MUL	NULL	

Diccionario de datos — captura 4

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
codigo	varchar(50)	YES		NULL	
nombre	varchar(180)	YES		NULL	
marca	varchar(100)	YES		NULL	
calidad	varchar(100)	YES		NULL	
tono_color	varchar(100)	YES		NULL	
presentacion	varchar(100)	YES		NULL	
precio	decimal(10,2)	YES		NULL	
existencia	int	YES		NULL	
stock_minimo	int	YES		NULL	
fecha_vencimiento	varchar(50)	YES		NULL	
id_categoria	int	YES		NULL	
id_proveedor	int	YES		NULL	
extra1	varchar(255)	YES		NULL	
extra2	varchar(255)	YES		NULL	

Diccionario de datos — captura 5

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id_proveedor	int	NO	PRI	NULL	auto_increment
nombre	varchar(150)	NO		NULL	
nit	varchar(30)	YES		NULL	
telefono	varchar(40)	YES		NULL	
correo	varchar(150)	YES		NULL	
direccion	varchar(255)	YES		NULL	

Diccionario de datos — captura 6

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id_rol	int	NO	PRI	NULL	auto_increment
nombre	varchar(80)	NO	UNI	NULL	
permisos	varchar(255)	YES		NULL	

Diccionario de datos — captura 7

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

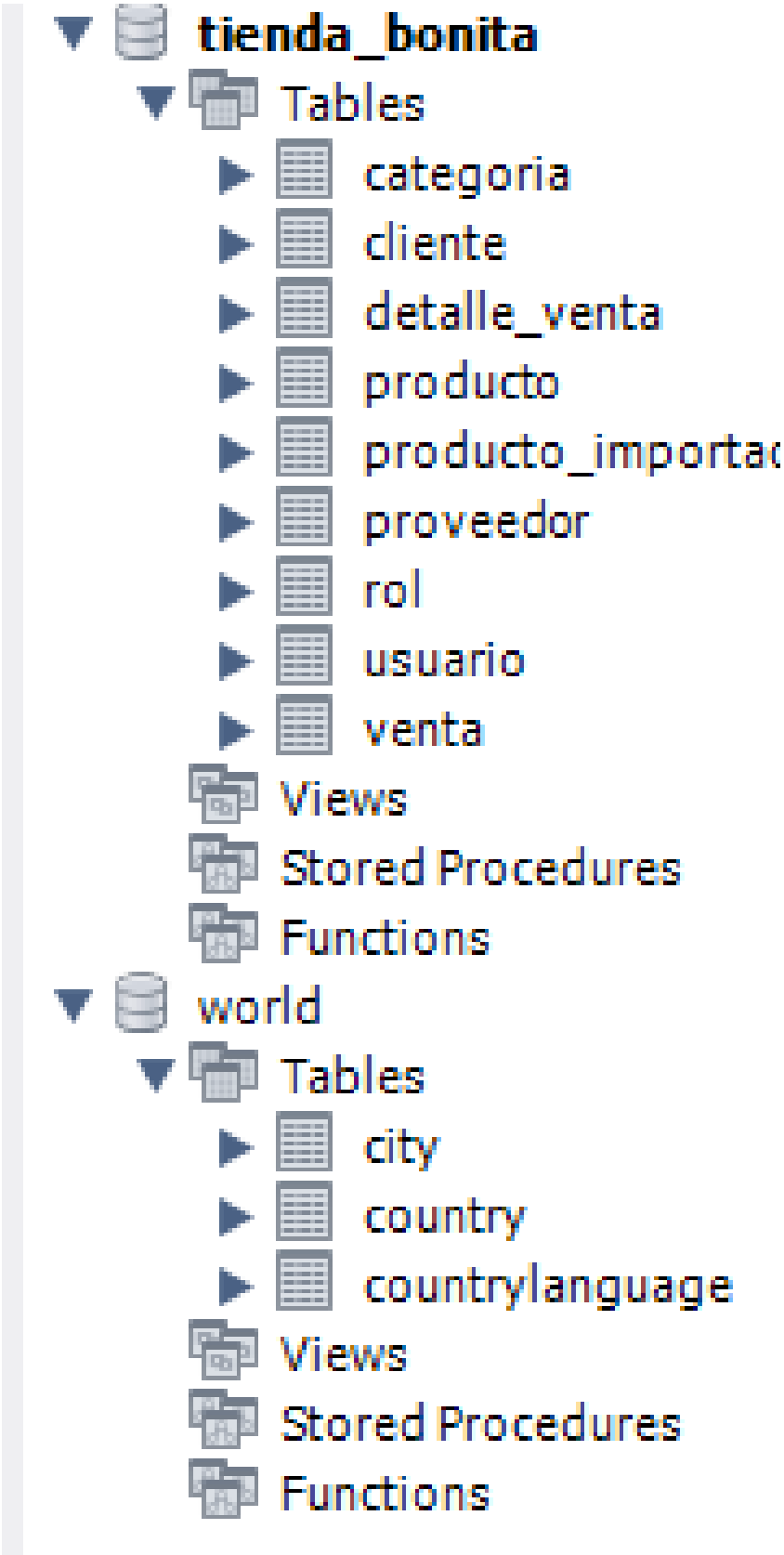
Field	Type	Null	Key	Default	Extra
id_usuario	int	NO	PRI	NULL	auto_increment
nombre	varchar(150)	NO		NULL	
usuario	varchar(80)	NO	UNI	NULL	
contrasena	varchar(255)	NO		NULL	
estado	tinyint(1)	NO		1	
id_rol	int	NO	MUL	NULL	

Diccionario de datos — captura 8

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id_venta	int	NO	PRI	NULL	auto_increment
fecha	date	NO	MUL	NULL	
id_cliente	int	NO	MUL	NULL	
id_usuario	int	NO	MUL	NULL	
total	decimal(12,2)	NO		0.00	
estado	varchar(30)	NO		Confirmada	

Diccionario de datos — captura 9



Diccionario de datos — captura 10

Enlaces y archivos de entrega

Peguen en los espacios siguientes los enlaces compartidos con permisos de lectura para el docente. Se conservan como campos editables para completar antes de entregar.

Entregable	Enlace
Código fuente del proyecto	https://drive.google.com/file/d/1q02QcPOfN-VM31dE68pSnUSfsC1H0P1S/view?usp=sharing
Script SQL / respaldo de base de datos	https://drive.google.com/file/d/1BMyJ3EWBqlO4DtLcJCaTllYhBAUErJxv/view?usp=sharing
Diagrama UML de clases y prototipo Draw.io	https://drive.google.com/file/d/1a1UD69Q5_NntAFukfpllFAp_MgKl188v/view
Diagrama entidad-relación / modelo relacional editable	https://drive.google.com/file/d/1Ifk3aeNFfbjfF_5IAldObIBUEjicoo5f/view
Carpeta general de evidencias	https://drive.google.com/drive/folders/1ssg1j5xTIxqgOdP5khePejs1eXylIJPd?usp=sharing
Carpeta de entrega de Proyecto	https://drive.google.com/drive/folders/1IXphfvD2xlPolydepENHtJO_hdnapk7l?usp=sharing

Videos individuales de demostración CRUD

Cada integrante debe agregar el enlace de su video individual, con cámara activa, explicando y demostrando un CRUD de la parte que desarrolló. Completen módulo y enlace antes de entregar.

Integrante	Carnet	Enlace del video
Angel Rodolfo Roblero Palma	0903-25-9479	https://drive.google.com/file/d/1l6ZwICcVYJU9iQbr2VlKLRLSy4upYGqw/view
Carol Andrea Gomez Dionicio	0903-25-23190	https://drive.google.com/file/d/1wLba6BUPVpLEw4kS18tgFgj1QFxqH0iZ/view?usp=sharing
Franshesca Sherlyn Cordon Orozco	0903-25-19546	https://drive.google.com/file/d/1Ifk3aeNFfbjfF_5IAldObIBUEjicoo5f/view
Zashary Adaly Fuentes Fuentes	0903-25-5941	https://drive.google.com/file/d/12-HrUzNckWctlqdnK1LRyL0DwzGpA2Fp/view?usp=sharing

Diccionario de datos en texto

Las definiciones siguientes se transcribieron de las capturas incluidas en el diccionario recibido. “NO” y “SÍ” indican si la columna permite NULL; PRI es clave primaria, UNI índice único, MUL índice no único y AI autoincremental. Verifiquen estas definiciones contra el script SQL final antes de entregar, ya que algunas capturas muestran diferencias con los diagramas.

Tabla categoria

Clasifica los productos.

Campo	Tipo	Nulo	Clave / índice	Predeterminado / extra
id_categoria	INT	NO	PRI	AUTO_INCREMENT
nombre	VARCHAR(100)	NO	—	—
descripcion	VARCHAR(255)	SÍ	—	NULL
estado	TINYINT(1)	NO	—	1

Relaciones: —

Tabla cliente

Almacena los datos de los clientes.

Campo	Tipo	Nulo	Clave / índice	Predeterminado / extra
id_cliente	INT	NO	PRI	AUTO_INCREMENT
nombre	VARCHAR(150)	NO	MUL*	—
nit	VARCHAR(30)	NO	—	—
telefono	VARCHAR(40)	SÍ	—	NULL
correo	VARCHAR(150)	SÍ	—	NULL
direccion	VARCHAR(255)	SÍ	—	NULL

Relaciones: venta.id_cliente referencia cliente.id_cliente. *La captura marca nombre como MUL; confirmar en SQL.

Tabla detalle_venta

Registra los productos, cantidades y valores de cada línea de venta.

Campo	Tipo	Nulo	Clave / índice	Predeterminado / extra
id_detalle	INT	NO	PRI	AUTO_INCREMENT
id_venta	INT	NO	MUL	—
id_producto	INT	NO	MUL	—
cantidad	INT	NO	—	—
precio	DECIMAL(10,2)	NO	—	—
subtotal	DECIMAL(12,2)	NO	—	—

Relaciones: id_venta → venta.id_venta; id_producto → producto.id_producto.

Tabla producto

Catálogo de productos y existencias.

Campo	Tipo	Nulo	Clave / índice	Predeterminado / extra
id_producto	INT	NO	PRI	AUTO_INCREMENT
codigo	VARCHAR(50)	NO	UNI	—
nombre	VARCHAR(180)	NO	—	—
marca	VARCHAR(100)	SÍ	—	NULL
calidad	VARCHAR(100)	SÍ	—	NULL
tono_color	VARCHAR(100)	SÍ	—	NULL
presentacion	VARCHAR(100)	SÍ	—	NULL
precio	DECIMAL(10,2)	NO	—	0.00
existencia	INT	NO	—	0
stock_minimo	INT	NO	—	0
fecha_vencimiento	DATE	SÍ	—	NULL
id_categoria	INT	NO	MUL	—
id_proveedor	INT	SÍ	MUL	NULL

Relaciones: id_categoria → categoria.id_categoria; id_proveedor → proveedor.id_proveedor.

Tabla producto_importacion

Tabla auxiliar de staging para importar productos. La captura no muestra clave primaria.

Campo	Tipo	Nulo	Clave / índice	Predeterminado / extra
codigo	VARCHAR(50)	SÍ	—	NULL
nombre	VARCHAR(180)	SÍ	—	NULL
marca	VARCHAR(100)	SÍ	—	NULL
calidad	VARCHAR(100)	SÍ	—	NULL
tono_color	VARCHAR(100)	SÍ	—	NULL
presentacion	VARCHAR(100)	SÍ	—	NULL
precio	DECIMAL(10,2)	SÍ	—	NULL
existencia	INT	SÍ	—	NULL
stock_minimo	INT	SÍ	—	NULL
fecha_vencimiento	VARCHAR(100)	SÍ	—	NULL
id_categoria	INT	SÍ	—	NULL
id_proveedor	INT	SÍ	—	NULL
extra1	VARCHAR(255)	SÍ	—	NULL
extra2	VARCHAR(255)	SÍ	—	NULL

Tabla proveedor

Almacena datos de contacto de los proveedores.

Campo	Tipo	Nulo	Clave / índice	Predeterminado / extra
id_proveedor	INT	NO	PRI	AUTO_INCREMENT
nombre	VARCHAR(150)	NO	—	—
nit	VARCHAR(30)	SÍ	—	NULL
telefono	VARCHAR(40)	SÍ	—	NULL
correo	VARCHAR(150)	SÍ	—	NULL
direccion	VARCHAR(255)	SÍ	—	NULL

Relaciones: producto.id_proveedor referencia proveedor.id_proveedor.

Tabla rol

Define perfiles de acceso.

Campo	Tipo	Nulo	Clave / índice	Predeterminado / extra
id_rol	INT	NO	PRI	AUTO_INCREMENT
nombre	VARCHAR(80)	NO	UNI	—
permisos	VARCHAR(255)	SÍ	—	NULL

Relaciones: usuario.id_rol referencia rol.id_rol.

Tabla usuario

Cuentas de acceso al sistema.

Campo	Tipo	Nulo	Clave / índice	Predeterminado / extra
id_usuario	INT	NO	PRI	AUTO_INCREMENT
nombre	VARCHAR(150)	NO	—	—
usuario	VARCHAR(80)	NO	UNI	—
contraseña	VARCHAR(255)	NO	—	—
estado	TINYINT(1)	NO	—	1
id_rol	INT	NO	MUL	—

Relaciones: id_rol → rol.id_rol; venta.id_usuario → usuario.id_usuario.

Tabla venta

Encabezado de las transacciones de venta.

Campo	Tipo	Nulo	Clave / índice	Predeterminado / extra
id_venta	INT	NO	PRI	AUTO_INCREMENT
fecha	DATE	NO	MUL	—
id_cliente	INT	NO	MUL	—
id_usuario	INT	NO	MUL	—
total	DECIMAL(12,2)	NO	—	0.00
estado	VARCHAR(30)	NO	—	Confirmada

Relaciones: id_cliente → cliente.id_cliente; id_usuario → usuario.id_usuario.

Conclusiones

1. La elaboración del diagrama de clases UML permitió identificar y organizar las entidades principales del sistema TIENDA BONITA, estableciendo sus atributos, métodos, responsabilidades y relaciones para representar la estructura lógica de la aplicación.
2. La aplicación de las convenciones UML, como el encapsulamiento, la visibilidad de los miembros y las multiplicidades, contribuye a mantener un modelo estructurado y comprensible para el desarrollo del sistema.
3. El prototipo de pantallas permitió representar visualmente los principales procesos de interacción del usuario, incluyendo el inicio de sesión, el registro de ventas y la gestión del inventario.
4. La relación entre el modelo UML y el prototipo visual proporciona una referencia para la implementación de las funcionalidades, facilitando la correspondencia entre la estructura de datos y las interfaces de usuario.
5. Los entregables desarrollados constituyen una base de diseño para orientar el desarrollo, mantenimiento y futuras mejoras del sistema TIENDA BONITA.